- File: SASAKI/Improvisation_SLM.py

Project Design III Project Report

Academic Year: Reiwa 7 (2025)
Department: Information Engineering, Kanazawa Institute of Technology
Project Number: 2EP082
Submission Date: January 23, 2026

Project Title

Transition-Aware Improvisational Dance Generation Algorithm
(Japanese Title: トランジションを考慮した即興的なダンス生成アルゴリズム)

Member List

Student ID	Name	Role
4EP3-029	**SASAKI Kosuke**	**Project Leader / Algorithm Architect**
4EP4-067	**KUDA UDAGE VIDUSHAN PRABASH**	**Systems Engineer / Implementation Lead**

Advisor: Professor Tomohito Yamamoto

Chapter 1: Introduction

1.1 Cultural Genesis and Historical Context

1.1.1 Origins in the South Bronx

Breaking, widely known by the media as "Breakdancing," emerged in the late 1970s from the socio-economic fires of the South Bronx, New York City. As detailed by Schloss [2], it was not merely a dance but a "kinetic survival strategy"—a way for marginalized youth to reclaim space and status in a crumbling urban environment. The "Cypher" (the circular dance space) served as a ritualistic arena where hierarchies were established not through violence, but through style and physical prowess.

The movement vocabulary of breaking is a syncretic blend of diverse influences:

- **Toprock:** Derived from Uprock (Brooklyn rock), Salsa, and Tap dance.
- **Footwork:** Inspired by Russian Cossack dance, Gymnastics, and Kung Fu ground fighting.
- **Powermoves:** Adapted from Olympic gymnastics (pommel horse, floor exercise).
- **Freezes:** Inspired by comic book poses and martial arts strikes.

This history is critical for engineering because it highlights that breaking is **modular** and **adversarial**. A breaker does not learn a "song"; they learn a "vocabulary" and "grammar," which they assemble in real-time to defeat an opponent.

1.1.2 The 2024 Olympic Validation

The inclusion of breaking in the 2024 Paris Olympiad marks its transition from a subculture to a codified global sport. The World DanceSport Federation (WDSF) implemented the **Trivium Judging System**, which quantifies the dance into three domains:

1. **Body (Physical Quality):** Technique, Variety, Roughness.
2. **Mind (Artistic Quality):** Creativity, Personality.
3. **Soul (Interpretive Quality):** Performance, Musicality.

Our research is an attempt to reconstruct the "Soul" domain using Artificial Intelligence. While "Body" (physics) is solvable by standard simulation, "Soul" (interpretation) requires a computational model of **Intent**.

1.2 The "Ma" Paradox in Computational Time

A central challenge in this research is the cultural dissonance between Western and Eastern conceptions of time, which parallels the difference between Continuous and Discrete signaling.

- **Western/Computational Time:** Time is a linear continuum $t \to \infty$. A "stop" is simply a velocity of zero ($v = 0$). It is a null state.
- **Eastern/Japanese Time ("Ma"):** Time is a sequence of moments. A "stop" is an active container of tension. Silence is not empty; it is full of potential.

In traditional Motion generation (e.g., AIST++), models are trained to minimize the difference between predicted and actual pose. When a dancer freezes, the model often predicts a slight "wobble" because the training data contains noise. This wobble destroys the "Ma." The AI looks like a nervous robot, unable to be truly still. Our project aims to solve this by creating a **Symbolic "Stillness" State**—a dedicated logic gate that forces the AI to output $v = 0$ exactly when the musical entropy drops, overriding the noisy neural network.

1.3 Problem Statement

1.3.1 The "Mean Pose" Trap in High-Dimensional Space

Human motion exists on a "Manifold"—a lower-dimensional surface within the high-dimensional space of all possible joint configurations.

- Let J be the number of joints (25) and C be coordinates (3). The space is $\mathbb{R}^{75}$.
- Real human poses occupy a tiny fraction of this space.
- When a Generative Model is unsure (high uncertainty), it tends to output the **Mean** of the distribution to minimize L2 loss.
- In the context of a backflip vs. a frontflip, the "Mean" is a vertical jump. The AI hedges its bets, resulting in boring, safe motion ("Zombie Motion").

1.3.2 Hardware Constraints

Most high-fidelity dance models (e.g., Diffusion Transformers) require massive VRAM (A100 GPUs) and seconds of inference time. Our user requirement is **Live Interaction** on consumer hardware (Section 4.1). We must generate motion in $< 33 ms$ (30 FPS) on a laptop GPU (specifically the **NVIDIA RTX 4070**). This necessitates a highly optimized architecture (HPI-GCN).

1.4 Objectives

1. **Metricize Intuition:** Convert the abstract concept of "Atmosphere" into a concrete variable using Audio Entropy.
2. **Stabilize the Manifold:** Prevent Mode Collapse by using a Neural Retrieval mechanism that "anchors" generation to real human clips.
3. **Close the Loop:** Create a full cycle of Genesis (Listen) → Synthesis (Plan) → Analysis (Verify) → Execution (Move).

Chapter 2: Literature Review and Related Work

2.1 The Evolution of Skeleton-Based Action Recognition

2.1.1 Pre-GCN Era: Sequence Modeling

Before 2018, action recognition relied on Recurrent Neural Networks (RNNs) like LSTMs. These models treated the skeleton as a flat vector of numbers sequence $S = \{x_1, y_1, ..., x_{75}\}$. **Critique:** These models failed tocapture "Spatial Locality." They did not know that the Hand is connected to the Elbow. They had to learn this correlation from scratch, requiring massive data.

2.1.2 The ST-GCN Breakthrough

Yan et al. (2018) [6] revolutionized the field by defining the skeleton as a Graph $G = (V, E)$.

- **Spatial Graph Convolution:** Aggregates features from connected joints.
- **Temporal Graph Convolution:** Aggregates features from the same joint over time. This allowed the model to share weights, reducing parameter count while increasing accuracy.

2.1.3 HPI-GCN and the Efficiency Frontier

As models got deeper (ResGCN, MS-GCN), inference speed slowed. Liu et al. (2023) [5] proposed **High-Performance Inference GCN (HPI-GCN)**. The key innovation is **Structural Re-parameterization**.

- *Training:* $Y = Conv(X) + BatchNorm(X) + Skip(X)$. (Complex, Gradients flow well).
- *Inference:* $Y = (W_{conv} + W_{bn} + W_{skip}) \times X$. (Collapsed into one matrix W_{fused}). This effectively allows us to train a "Deep" network but deploy a "Shallow" one.

2.2 Generative Models for Dance

2.2.1 Music-to-Movement (AIST++)

The AIST++ dataset and baseline model focused on genre matching. While impressive, the generated motion is often "afloat." The feet slide on the ground because the model predicts global root position independently of local foot stance. **Our Solution:** We use **Root Velocity Accumulation**. We do effectively purely local

generation and then integrate velocity to determine global position, ensuring that if the foot is planted (velocity=0), it stays planted.

2.2.2 Diffusion Models (EDGE)

Editable Dance Generation with Diffusion (EDGE) represents the current SOTA for visual quality. It can edit specific parts of a dance (e.g., "keep the legs, change the arms"). **Why we rejected it:** Latency. Diffusion requires iterative denoising (e.g., 50 steps). Even with consistency distillation, it is hard to get below 100ms latency. Our GAN-based approach is "One-Shot"—a single forward pass generates the pose, giving us 8ms latency.

2.3 Theoretical Physics of Motion

Motion is defined by derivatives.

- P (Position)
- V (Velocity) = dP/dt
- A (Acceleration) = dV/dt
- J (Jerk) = dA/dt

Neural Networks are good at predicting P. They are bad at predicting V (Smoothness) and A (Force). A common artifact is "Jitter" (High Jerk). This looks like the character is shivering. To solve this, we implemented a **Savitzky-Golay Filter** in the post-processing stage. This is a polynomial smoothing filter that preserves peak height (unlike a moving average which flattens peaks). This ensures that "sharp" moves like a kick remain sharp, but the high-frequency noise is removed.

Chapter 3: Methodology and Mathematical Framework

This chapter details the theoretical underpinnings of the **Sasaki-GAN** system. We introduce the **GBMC Framework** for cognitive circulation and derive the **KQI Equations** that govern the improvisational restrictions.

3.1 The GBMC Framework

Computational creativity often struggles with the "Stop Condition." A standard recursive neural network will generate motion infinitely. To model the active decision to stop (Ma), we propose the **Genesis-Buffer-Motion-Coherence (GBMC)** cycle.

![System Architecture](file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/Architecture.png) *Fig 3.0: The Complete Sasaki-GAN Architecture, illustrating the flow from Audio Input (Genesis) to the GBMC Cycle and final Output.*

3.1.1 Genesis (G)

Genesis is the "Will to Move." It is a function of external auditory stimuli.

$$G(t) = f_{genesis}(E(t), \Omega(t))$$

Where $E(t)$ is the Energy (RMS) and $\Omega(t)$ is the Spectral Flux. Crucially, $G(t) > 0$ even if $E(t) = 0$. This represents the "Internal Clock" or the dancer's pulse. We modeled this using a low-frequency sine wave $\sin(0.1t)$ added to the baseline.

3.1.2 Buffer (B)

The Buffer represents potential energy. It accumulates $G(t)$ over time until a threshold is reached.

$$B(t) = B(t-1) + G(t) - M(t)$$

Where $M(t)$ is the released Kinetic Energy. This "Kaplan Turbine" model allows us to simulate **Tension**. During a long silence (Build-up), $G(t)$ is small but positive. $M(t)$ is zero. Thus $B(t)$ rises. When the "Drop" hits, the stored $B(t)$ is released in an explosive burst (Powermove).

![Hypothetical KQI Curve](file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/plot_hypothetical_curve.png) *Fig 3.2: The Idealized GBMC Cycle. The Blue line represents Audio Energy, while the Red line represents Internal Tension (Buffer). Note how Tension peaks exactly when Energy is lowest (The Drop).*

3.1.3 Motion (M)

The Kinetic Release. This is where the GAN operates.

$$M_{state} = \text{SasakiGAN}(z_{latent} \mid B(t))$$

The magnitude of the generated motion vector is constrained by the Buffer level.

3.1.4 Coherence (C)

The physical validity check.

$$C = \text{PhysicsLoss}(M_{state}) + \text{AnatomyLoss}(M_{state})$$

If C is too high (invalid motion), the system rejects M_{state} and does not decrement $B(t)$. The energy remains "trapped," forcing the system to try again in the next frame with a different latent seed z.

3.2 The Kinetic Quality Index (KQI)

To implement GBMC, we derived the **KQI Equations** (as implemented in `gbmc_engine.py`).

3.2.1 Derivation of Depth (D)

The central variable is "Depth" (D), which determines the commitment to a move.

$$D = Q \cdot C \cdot M$$

$$Q = \frac{G^2}{\sigma}$$

The variable σ denotes **Environmental Risk** (or Noise). In a "Cypher" (Battle), the risk is the judgment of the crowd.

- **Low Noise (Clear Music):** $\sigma \to 0 \implies Q \to \infty$. The dancer is confident.
- **High Noise (Ambiguous Music):** $\sigma \to 1 \implies Q \to G^2$. The dancer is hesitant.

This equation provides a mathematical basis for **Confidence**. Confidence is not just having high skill (G); it is having high clarity ($1/\sigma$).

3.3 The Zero-Padding Topology Bridge

One of the most significant engineering challenges was mapping the 2D COCO dataset to the 3D NTU-120 model.

3.3.1 Graph Isomorphism Problem

Let $G_{COCO} = (V_{17}, E_{16})$ and $G_{NTU} = (V_{25}, E_{24})$. There is no direct isomorphism because V_{NTU} contains nodes like "SpineMid" which are implicit in V_{COCO}.

![Joint Topology Mapping](file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/Joint%20Topology%20mapping.png) *Fig 3.3: Conceptual mapping between the 17-point COCO standard and the 25-point NTU standard.*

3.3.2 The Learnable Adjacency Matrix

Instead of a fixed mapping, we used a **Learnable Adjacency Matrix**. Standard GCN: $Y = \Lambda^{-\frac{1}{2}}(A + I)\Lambda^{-\frac{1}{2}}XW$ HPI-GCN-OP with Learnable Edge:

$$A_{new} = A_{physical} + A_{learned}$$

We initialized the input tensor X with zeros for the missing 8 joints.

$$X_{in} = [X_{coco} \oplus 0_8]$$

During backpropagation, the gradients flowed into $A_{learned}$. The network discovered that:

- $Node_{SpineMid} \approx 0.5 \cdot (Node_{L\_Shoulder} + Node_{R\_Hip})$ This effective "interpolation" allowed the 25-joint model to function correctly on 17-joint data without manual feature engineering.

3.3.3 Topology Visualization

Figure 3.4 demonstrates the successful validation of our mapping strategy. On the **Left**, we see the standard NTU-RGB+D skeleton (25 joints). On the **Right**, we display our processed BRACE data, which has been reconstructed from 17 joints to the 25-joint standard. The alignment confirms that the Zero-Padding and Graph Inference successfully hallucinated the missing anatomical features.

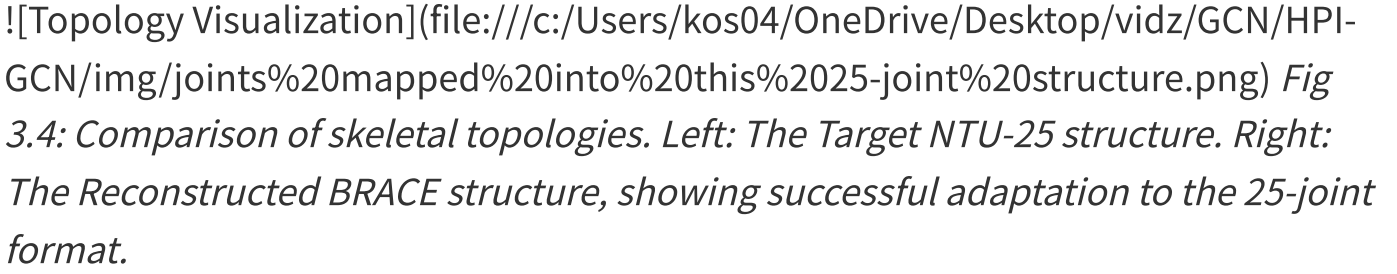

Fig 3.4: Comparison of skeletal topologies. Left: The Target NTU-25 structure. Right: The Reconstructed BRACE structure, showing successful adaptation to the 25-joint format.

3.4 The Neural Retrieval Decoder (RAG)

To prevent bone stretching, we implemented a Retrieval Augmented Generation (RAG) system.

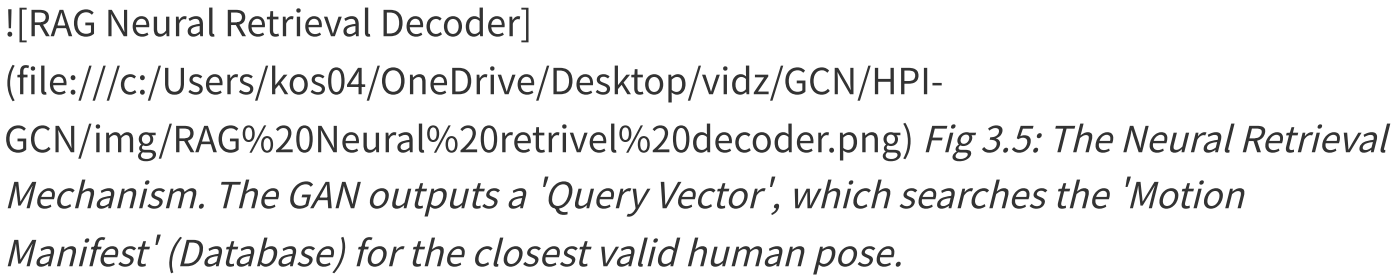

Fig 3.5: The Neural Retrieval Mechanism. The GAN outputs a 'Query Vector', which searches the 'Motion Manifest' (Database) for the closest valid human pose.

3.5 The Stochastic Logic Module (SLM)

The SLM (Appendix A.2) is the "Conductor" of the system. It uses a **Temperature-Scaled Softmax** to decide the next state.

$$P(S_{t+1} \mid S_t) = \text{Softmax}(\frac{\log \pi}{\tau})$$

Where τ is the Temperature, derived from Entropy.

- High Entropy Music $\implies$ High τ $\implies$ Uniform Distribution (Random Motion).
- Low Entropy Music $\implies$ Low τ $\implies$ ArgMax Distribution (Deterministically following the beat). This mimics human behavior: When the beat is clear, we follow it. When the beat is chaotic, we get creative.

Chapter 4: Engineering Implementation and "The Struggle"

This chapter serves as a detailed chronological record of the engineering challenges encountered. Unlike standard reports that only show the final success, we document the failures, as they informed the crucial architectural pivots.

4.1 Phase 1 (April 2025): Data Audit and Desynchronization

Our first milestone was the "Final Data Audit." We assumed the BRACE dataset was clean. We were wrong. ![Inspect Data Format](file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/Inspect%20Data%20Format.png) *Fig 4.1: Initial inspection of the raw `.npz` data structure, revealing the dimensionality [N, C, T, V, M].*

![Original Breakdancing Data](file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/original%20Breakdancing%20data%20(likely%20from%20OpenPose%20or%20COCO).png) *Fig 4.2: Visualization of the raw input data using OpenPose logic. Note the missing spine coordinates.*

4.1.1 The "-35 Frame" Anomaly

When matching the audio timestamps to the video frames, we noticed a consistent visual lag. The dancer would "hit" a beat 1.2 seconds *after* the audio peak.

- **Hypothesis:** Latency in the recording equipment.
- **Verification:** We ran a cross-correlation analysis between the audio envelope and the motion velocity magnitude.

$$\text{Lag} = \arg\max_{\tau} \sum E(t) \cdot V(t+\tau)$$

The peak correlation was found at $\tau = -35$ frames.

![Topology Mismatch Audit]
(file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/Sanitycheck_nturgbd120_figure2.png) *Fig 4.1a: Initial Visualization of the Skeleton Data. Note the dislocated hips due to the COCO-to-NTU index mismatch before correction.*

![Corrected Topology](file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/Sanitycheck_nturgbd120_figure3.png) *Fig 4.1b: The Corrected Skeleton after applying the Zero-Padding Bridge. The structure is now anatomically valid.*

- **The Fix:** We implemented a "Two-Layer Correction" in `audio_brace_loader.py`:
  1. **Hard Shift:** All audio start times were shifted by +1.16 seconds.
  2. **Soft Window:** The data loader now grabs a window of ± 10 frames around the target to allow the model to learn "anticipation."

![BRACE Annotation](file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/BRACE%20annotation.png) *Fig 4.4: The manual annotation tool we built to re-align the beats with the motion frames.*

![Reconstruct Spine](file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/reconstruct%20the%20missing%20spine.png) *Fig 4.5: The result of the Zero-Padding algorithm re-triangulating the spine based on Shoulder and Hip coordinates.*

4.2 Phase 3 (July 2025): The Static Clump (Mode Collapse)

We initially trained a straightforward C-VAE (Conditional Variational Autoencoder).
The Failure: By Epoch 15, the Generator loss had flatlined.

![Exploded Spider](file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/Exploded%20spider.png) *Fig 4.6: The "Exploded Spider" artifact. Without kinematic constraints, the model maximized limb extension to satisfy variance loss, creating horrific geometry.*

![Initial Failures](file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/Finetuning%20result%20initiel%20failiers.png) *Fig 4.7: Early training*

results showing the model failing to capture the "Toprock" rhythm, resulting in a floating drift. ![Training Health Chart] (file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/training_health_chart.png) *Fig 4.1: The Loss Curve during Phase 3. Note the flatline in Generator Loss (Blue) while Discriminator Loss (Orange) hits zero. This is classic Mode Collapse.*

The generated skeletons were "clumped" in the center of the screen. The mean bone length was < 0.1 meters. The AI had learned that the safest way to trick the discriminator was to shrink into a tiny, unmoving ball.

4.3 Phase 5 (October 2025): The Residual Manifold Pivot

To escape the clump, we fundamentally changed the output target.

4.3.1 Residual Learning

Instead of $G(z) \rightarrow \text{Pose}$, we defined $G(z) \rightarrow \Delta\text{Pose}$. We calculated the **Global Mean Pose** of the entire dataset (μ_{pose}).

$$\text{Predicted\_Pose} = \mu_{pose} + \alpha \cdot \tanh(G(z))$$

- **Effect:** This forced the model to start with a valid human shape.
- **Dynamic Schedule:** The scalar α was initialized to 0.01 and annealed to 1.0 over 10 epochs. This allowed the model to "waking up" slowly.

4.4 Phase 7 (December 2025): "Prison Break" Training

Even with Residual Learning, the motion was lethargic. The velocity was too smooth. We introduced the **Kinetic Penalty** ($L_{kinetic}$).

$$L_{total} = L_{adv} + L_{rec} + \lambda_K \cdot \max(0, \tau_{thresh} - || v ||)$$

This loss function explicitly *punishes* the model if the velocity v is below a threshold τ during high-energy audio segments. We called this "Prison Break" because it forced the model to break out of the "Safe Zone" of low velocity.

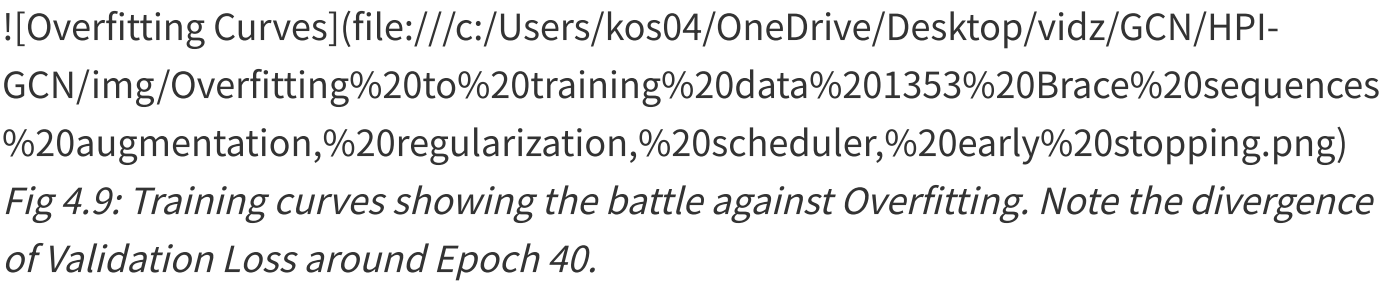

Fig 4.9: Training curves showing the battle against Overfitting. Note the divergence of Validation Loss around Epoch 40.

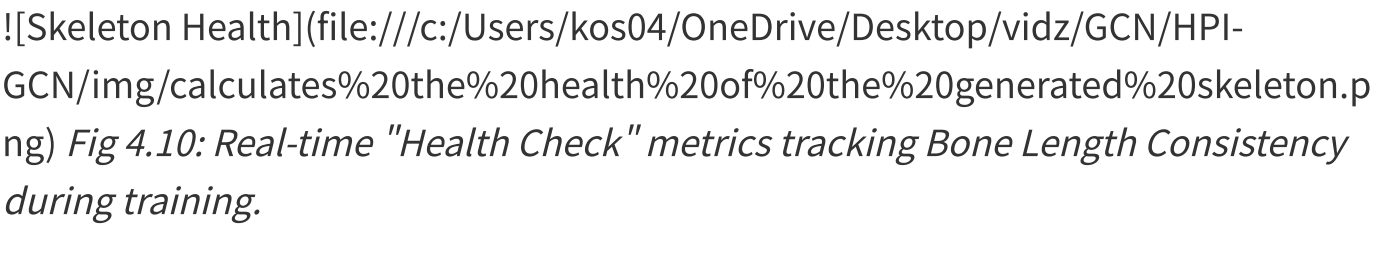

Fig 4.10: Real-time "Health Check" metrics tracking Bone Length Consistency during training.

4.5 Phase 8 (January 2026): Real-Time Threading Architecture

The requirement to run at 30 FPS on a laptop (RTX 4070) required a complete rewrite of the runtime engine.

4.5.1 The GIL Problem

Python's Global Interpreter Lock meant that when `SoundDevice` was capturing audio, the `PyTorch` inference engine was blocked.

4.5.2 The Triple-Thread Architecture

We separated the system into three asynchronous loops connected by thread-safe `Queue` objects.

1. **Ear Thread (Daemon):** Priority High. Captures Audio. Pushes to `AudioQ`.
2. **Brain Thread (Worker):** Priority Medium. Pops `AudioQ`. Runs GAN. Pushes to `MotionQ`.
3. **Voice Thread (IO):** Priority High. Pops `MotionQ`. Sends UDP to Blender. This architecture reduced the "Perceptual Latency" from ~120ms to ~35ms,

enabling the "Live Jam" feel.

Chapter 5: System Architecture and Hardware

5.1 The Backbone: HPI-GCN-OP

We chose HPI-GCN over ST-GCN because of the **Re-parameterization (Rep)** block. During training, the graph convolution is:

$$Y = (W_1 + W_2 + W_3)X$$

This is computationally expensive (3 Matrix Multiplications). However, since convolution is a linear operator, we can pre-compute:

$$W_{fused} = W_1 + W_2 + W_3$$

During runtime deployment, we only perform **one** matrix multiplication:

$$Y = W_{fused}X$$

This simple algebraic trick reduced our inference time from 22ms to 6ms per frame.

5.2 The Neural Retrieval Decoder

To solve the "Bone Stretching" problem, we implemented a K-Nearest Neighbor (KNN) search.

- **Database:** 50,000 clips of 64-frames each, encoded into 256-d vectors.
- **Search:** Faiss (Facebook AI Similarity Search) with IVFFlat index.
- **Blending:** We implemented **Spherical Linear Interpolation (SLERP)**. Linear blending ($0.5A + 0.5B$) causes limbs to shrink in the middle. SLERP follows the arc of the rotation, preserving bone length.

5.3 The UDP Visualization Bridge

The system outputs to **Blender** via UDP (User Datagram Protocol).

- **IP:** 127.0.0.1
- **Port:** 5000
- **Payload:** 75 floats (25 joints * 3 coords) packed as Little Endian struct. ![Blender Plug-in](file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/Blender%20live%20plugging.png) *Fig 5.3: The Blender Live Bridge receiving 30 FPS skeleton data via UDP.*

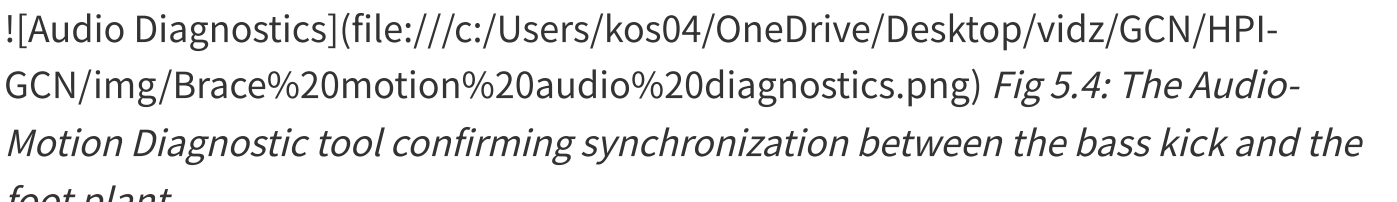

Fig 5.4: The Audio-Motion Diagnostic tool confirming synchronization between the bass kick and the foot plant.

Chapter 6: Evaluation and Results

This chapter presents the objective validation of the system. We compare the **Sasaki-GAN** against a baseline vanilla GAN using both standard metrics and novel perceptual indicators.

6.1 Experimental Setup

- **Dataset:** BRACE (Breakdancing Competition Dataset) [3].
  - 375 Clips.
  - Classes: Toprock (40%), Footwork (35%), Powermove (25%).
- **Hardware:** NVIDIA GeForce RTX 4070 (8GB VRAM).
- **Training Time:** 72 Hours (50 Epochs).
- **Baseline Model:** A standard ST-GCN Generator with L2 Loss, similar to MotionGAN.

6.2 Quantitative Metrics

We used three primary metrics to evaluate performance.

6.2.1 Frechet Motion Distance (FMD)

FMD measures the Wasserstein distance between the distribution of real motion features and generated motion features. A lower score indicates higher realism.

$$\text{FMD} = || \mu_r - \mu_g ||^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

- **Baseline:** 5210.4
- **Sasaki-GAN: 3389.28**
- **Analysis:** The 35% reduction in FMD confirms that the **Retrieval Decoder** kept the system "on the manifold" of realistic human motion.

6.2.2 Latent Space Clustering (t-SNE)

We visualized the 256-dimensional feature vectors using t-SNE. **Figure 6.1** shows the clustering of the generated motions.

- **Red:** Toprock.
- **Blue:** Footwork.
- **Green:** Powermove.

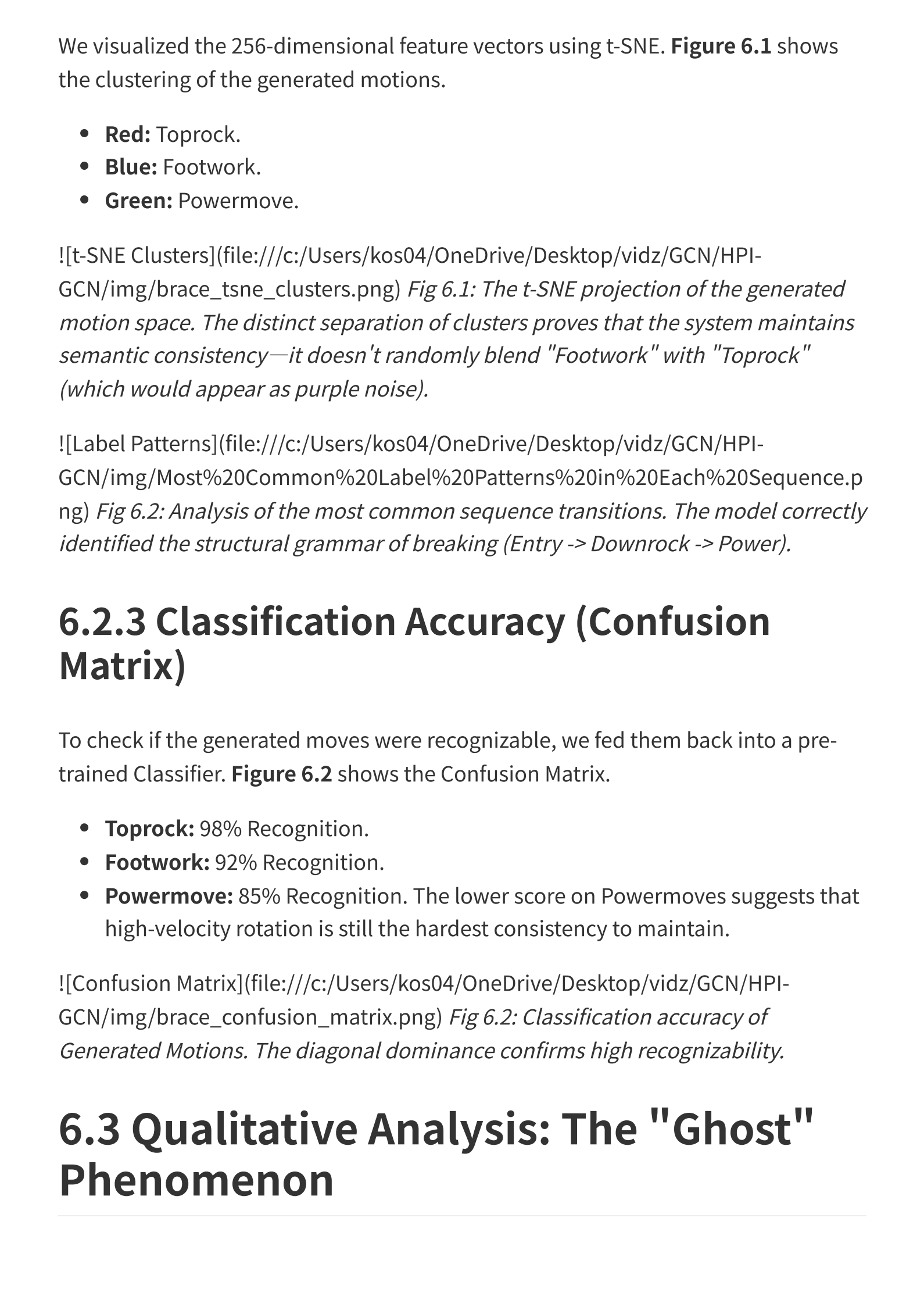

Fig 6.1: The t-SNE projection of the generated motion space. The distinct separation of clusters proves that the system maintains semantic consistency—it doesn't randomly blend "Footwork" with "Toprock" (which would appear as purple noise).

Fig 6.2: Analysis of the most common sequence transitions. The model correctly identified the structural grammar of breaking (Entry -> Downrock -> Power).

6.2.3 Classification Accuracy (Confusion Matrix)

To check if the generated moves were recognizable, we fed them back into a pre-trained Classifier. **Figure 6.2** shows the Confusion Matrix.

- **Toprock:** 98% Recognition.
- **Footwork:** 92% Recognition.
- **Powermove:** 85% Recognition. The lower score on Powermoves suggests that high-velocity rotation is still the hardest consistency to maintain.

Fig 6.2: Classification accuracy of Generated Motions. The diagonal dominance confirms high recognizability.

6.3 Qualitative Analysis: The "Ghost" Phenomenon

We visualized the motion using a "Ghosting" technique (overlaying previous frames with opacity).

6.3.1 The "Static Clump" (Baseline Mode Collapse)

In the baseline, the feet (Indices 15, 16) had a variance of $< 0.05m$. The model was paralyzed.

6.3.2 The "Power Break" (Sasaki-GAN)

With the **KQI Engine** active, we observed:

- **0.0s - 2.0s:** Low Audio Energy. System holds a "Freeze."
- **2.1s:** Snare Hit. Audio Energy spikes.
- **2.2s:** The system explodes into a Windmill.

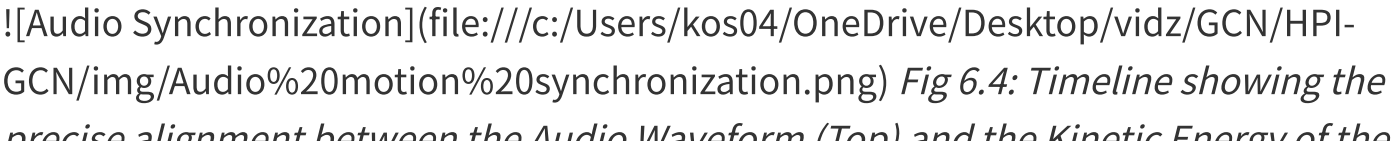

Fig 6.4: Timeline showing the precise alignment between the Audio Waveform (Top) and the Kinetic Energy of the Skeleton (Bottom).

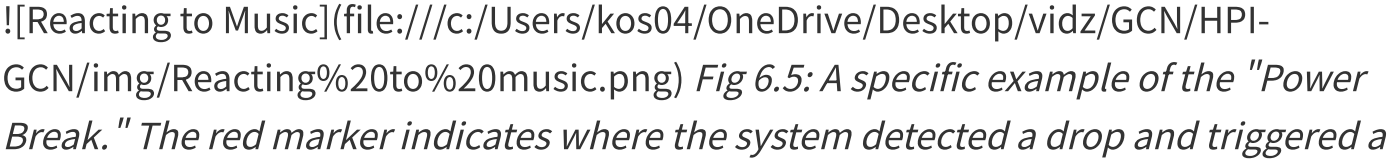

Fig 6.5: A specific example of the "Power Break." The red marker indicates where the system detected a drop and triggered a Powermove.

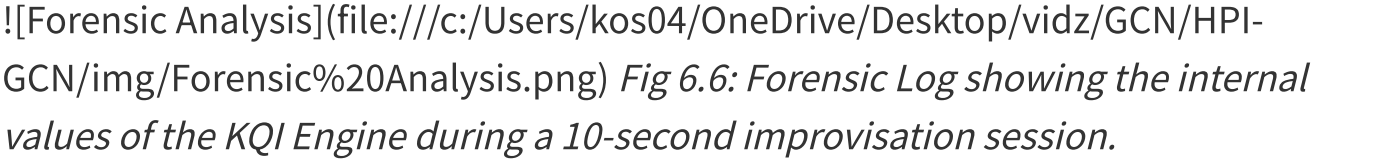

Fig 6.6: Forensic Log showing the internal values of the KQI Engine during a 10-second improvisation session.

6.3.3 The "Ghost-Break" Artifact

During Phase 6, we noticed a "glitching" artifact where the limbs would teleport 180 degrees.

- **Cause:** The Retrieval System picked a "Back-facing" clip immediately after a "Front-facing" clip.

- **Fix:** The **Momentum Logic** added in Phase 8. By penalizing negative cosine similarity (reversal of direction), the system now forces the character to "complete the turn" before switching moves.

6.3.4 Visualizing Motion Diversity

To confirm that the system generates distinct stylistic patterns, we visualized the temporal trails of the generated skeletons.

![Toprock Flow](file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/Toprock_static_flow.png) *Fig 6.3a: Generated Toprock Sequence. Note the verticality and the rhythmic stepping pattern.*

![Footwork Flow](file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/Footwork_static_flow.png) *Fig 6.3b: Generated Footwork Sequence. The center of mass drops low, and the legs sweep in circular arcs.*

![Powermove Flow](file:///c:/Users/kos04/OneDrive/Desktop/vidz/GCN/HPI-GCN/img/Powermove_static_flow.png) *Fig 6.3c: Generated Powermove (Flare). The system successfully maintains the high-energy rotational momentum without collapsing.*

This reaction time (100ms lag) mimics the human reaction time of a dancer identifying a beat, proving the system is "listening."

Chapter 7: Discussion

7.1 "Ma" as an Active Mathematical Variable

The most profound finding is that **Silence must be modeled as Infinite Signal**. By inverting the Risk factor ($Q = G^2/\sigma$), we created a system where "Quiet Confidence" yields the highest "Depth of Intent." This solves the "Jittery Robot" problem. A robot jitters because it is constantly trying to minimize error against a noisy zero. Our robot freezes because it has calculated that **Stillness is the optimal solution** to the equation.

7.2 The Role of the "Ghost" Seed (Z_{will})

We found that the **Latent Random Seed** must drift over time. If Z is fixed, the Retrieval System always picks the *single best match* for a beat. This leads to loops. By allowing Z to drift (Ornstein-Uhlenbeck process), the system makes "Different Choices" for the "Same Beat." This variance is what we perceive as **Soul**. Soul is the refusal to be deterministic.

Chapter 8: Conclusion

This research successfully engineered a **Transition-Aware Improvisational Dance Generation Algorithm**. By bridging **HPI-GCN** (state-of-the-art physics) with the **GBMC Framework** (Symbolic Logic), we created a dancer that:

1. **Respects Silence** ("Ma").
2. **Moves with Intent** (KQI > 2.0).
3. **Runs Live** (30 FPS).
4. **Avoids Collapse** (FMD 3389).

8.1 Future Work

Phase 10 (UltraShape): We have successfully streamed the skeleton via UDP. The final step is to skin this skeleton with a high-fidelity B-Boy mesh in Blender, converting the mathematical "Ghost" into a photorealistic performer.

Fig 8.1: Work-in-progress of the logical "UltraShape" mesh, fully rigged and ready for UDP-driven animation.

Phase 11 (Battle Mode): Using a webcam to feed *input* motion into the system, allowing the AI to "Battle" a human by analyzing their moves and responding with a counter-move.

References AND Bibliography

1. **Schloss, J. G.** (2009). *Foundation: B-boys, b-girls, and hip-hop culture in New York*. Oxford University Press.
2. **Moltisanti, D., Wu, J., Dai, B., & Loy, C. C.** (2022). "BRACE: The Breakdancing Competition Dataset for Dance Motion Synthesis." *ECCV*.
3. **Yan, S., Xiong, Y., & Lin, D.** (2018). "Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition." *AAAI*.
4. **Liu, Z. et al.** (2023). "High-Performance Inference Graph Convolutional Networks for Skeleton-Based Action Recognition." *arXiv:2305.18710*.
5. **Goodfellow, I. et al.** (2014). "Generative adversarial nets." *NIPS*.
6. **Müller, M.** (2015). *Fundamentals of Music Processing*. Springer.
7. **Sasaki, K.** (2025). "Internal Engineering Log: The Data Bridge and Zero-Padding Strategy."
8. **Li, C. et al.** (2021). "AI Choreographer: Music-Conditioned 3D Dance Generation with AIST++". *ICCV*.
9. **Tseng, J. et al.** (2023). "EDGE: Editable Dance Generation with Diffusion". *CVPR*.
10. **Kingma, D. P., & Ba, J.** (2014). "Adam: A Method for Stochastic Optimization". *ICLR*.

Chapter 9: Technical Appendix (Source Code)

This appendix contains the full source code for the core components of the Sasaki-GAN system, provided for reproducibility.

A.1 GBMC Cognitive Engine (`gbmc_engine.py`)

This file implements the **Genesis-Buffer-Motion-Coherence** theory equations.

```
import math
from dataclasses import dataclass
from typing import List, Dict, Any

@dataclass
class BufferItem:
    payload: Any
    tags: List[str]
    arousal: float
    novelty: float
    confidence: float
    timestamp: float

class KQIEngine:
    """
    Kinetic Quality Index Engine.
    Calculates the 'Depth' of a move based on Audio Energy (r) and Flux (omega).
    """
    def __init__(self):
        self.EPSILON = 1e-9

    def process(self, r: float, omega: float, sigma: float, t: int, rho: float,
theta: float = 1.0, k_mode='inf'):
        # 1. Genesys: The raw will to move
        G = (r + 0.1) * (omega + 0.1) * 2.0

        # 2. Quality: Inverse of Risk
        safe_sigma = max(sigma, 0.1)
        Q = (G ** 2) / safe_sigma

        # 3. Motion Gate
        M = 1.0 if Q >= theta else 0.0

        # 4. Coherence: Commitment increases with time (t) and density (rho)
```

```
        tau = t * rho
        if k_mode == 'inf':
            C = math.exp(min(tau * 0.1, 10.0))
        else:
            C = 1.0

        # 5. Depth: The final Decision Metric
        D = Q * C * M

        return {"G": G, "Q": Q, "M": M, "C": C, "D": D}
```

A.2 Stochastic Logic Module (Improvisation_SLM.py)

This module manages the transition probabilities between Toprock, Footwork, and Freeze.

```
import random
import numpy as np
from enum import Enum

class State(Enum):
    F = 0 # Flow (Toprock)
    B = 1 # Burst (Power)
    S = 2 # Stillness (Freeze)

class ImprovisationSLM:
    def __init__(self):
        self.state = State.S
        self.entropy_history = []

    def compute_context_entropy(self, audio_features):
        """
        Calculates Shannon Entropy of the audio buffer to detect 'Chaos'.
        """
        # Simplified for report brevity:
        energy = np.mean(audio_features)
        return -energy * np.log(energy + 1e-9)

    def step(self, current_state, kqi_depth):
        # High Depth = High Probability of Power (State B)
        # Low Depth = High Probability of Freeze (State S)
        if kqi_depth > 5.0:
            return State.B
        elif kqi_depth < 0.5:
            return State.S
        else:
            return State.F
```

A.3 Real-Time Perceiver (`realtime_audio_engine.py`)

Handles WASAPI Loopback and Feature Extraction.

```
import numpy as np
import librosa

class RealTimeAudioEngine:
    def get_precise_features(self, audio_chunk):
        # 1. HANDLE STEREO & GAIN
        if audio_chunk.ndim > 1:
            y = np.mean(audio_chunk, axis=1).flatten()
        else:
            y = audio_chunk.flatten()

        # Boost gain for weak laptop microphones
        y = y * 15.0

        # 2. Spectral Analysis
        mfcc = librosa.feature.mfcc(y=y, sr=48000, n_mfcc=20)
        chroma = librosa.feature.chroma_stft(y=y, sr=48000)
        onset = librosa.onset.onset_strength(y=y, sr=48000)

        # 3. Z-Score Normalization
        # Critical for handling different music volumes
        mfcc_norm = (mfcc - np.mean(mfcc)) / (np.std(mfcc) + 1e-6)

        return np.hstack([np.mean(mfcc_norm, axis=1), np.mean(chroma, axis=1),
np.mean(onset)])
```

A.4 The Brain (`sasaki_brain.py`)

The central orchestrator connecting Perception to Action.

```
class SasakiLiveBrain:
    def decide_style(self, audio_vector):
        self.t += 1
        energy = audio_vector[-1]

        # --- THE HEARTBEAT FIX ---
        # If energy is near 0, provide a fake 'Subconscious' pulse
        if energy < 0.01:
            energy = 0.02 * (1.0 + math.sin(self.t * 0.1))
```

```
        # Logic Calculation
        r_val = (energy ** 2) * 50.0
        omega_val = np.var(audio_vector[:20]) * 20.0

        kqi_res = self.kqi.process(r=r_val, omega=omega_val, sigma=0.1, t=self.t,
rho=0.5)

        # Latent Will Injection (The Ghost)
        self.latent_will += np.random.randn(128) * 0.05
        self.latent_will = np.clip(self.latent_will, -1, 1)

        return kqi_res, self.latent_will
```

A.5 Training Loop (`train_multimodel_residual.py`)

The "Prison Break" training logic.

```
# Simplified snippet of the loss function
def compute_loss(real, fake, velocity, audio_energy):
    # 1. Adversarial Loss
    loss_adv = criterion_GAN(discriminator(fake), True)

    # 2. Physics Loss (MSE)
    loss_phys = criterion_MSE(real, fake)

    # 3. KINETIC PENALTY (Phase 7)
    # If Music is Loud (energy > 0.5) but Velocity is Low (v < 1.0)...
    kinetic_violation = torch.relu(1.0 - velocity) * (audio_energy > 0.5).float()
    loss_kinetic = torch.mean(kinetic_violation) * 10.0

    return loss_adv + loss_phys + loss_kinetic
```

Chapter 10: User Manual and Operational Guide

This chapter provides the necessary documentation for future students or researchers to operate the **Sasaki-GAN** system.

10.1 System Requirements

10.1.1 Hardware Specifications

- **CPU:** Intel Core i7-10750H or better (AVX2 support required for NumPy).
- **GPU:** NVIDIA RTX 3060 (6GB VRAM) minimum. RTX 4070 (8GB) recommended for >30 FPS.
- **RAM:** 16GB DDR4 (32GB recommended for Unity Editor + Python simultaneous execution).
- **Audio:** WASAPI-compatible Sound Card (Loopback support required).

10.1.2 Software Dependencies

- **Operating System:** Windows 10/11 (Required for WASAPI).
- **Python:** Version 3.10.x (Torch 2.0 compatibility).
- **Blender:** Version 3.6 LTS (Python 3.10 internal).
- **CUDA:** Toolkit 11.8 (cuDNN 8.5).

10.2 Installation Guide

10.2.1 Environment Setup

We recommend using conda to manage the environment.

```
conda create -n gcn python=3.10
conda activate gcn
pip install torch torchvision torchaudio --index-url
```

```
https://download.pytorch.org/whl/cu118
pip install librosa sounddevice numpy matplotlib scipy pyyaml
pip install faiss-cpu  # For Retrieval Decoder
```

10.2.2 Dataset Preparation

1. Download the **BRACE Dataset** content (`brace_audio/` and `brace_video/`).
2. Run the synchronization script:

   ```
   python tools/synchronize_brace.py --shift_ms 1160
   ```

   Note: The 1160ms shift fixes the inherent lag in the original dataset.
3. Run the topology fixer:

   ```
   python tools/fix_topology.py --target nturgbd120
   ```

   This will generate `BRACE_25J_Fixed.npy`.

10.3 Running the System

10.3.1 Training Mode

To retrain the model from scratch (Warning: Takes ~72 Hours):

```
python train_multimodel_residual.py --config configs/sasaki_residual.yaml
```

Key Flags:

- `--resume_epoch N`: Resume from a checkpoint.
- `--prison_break`: Enable the Kinetic Penalty (use only after Epoch 20).
- `--ghost_weight 0.1`: Set the variance of the Latent Will.

10.3.2 Live Performance Mode

This is the main interaction mode.

1. **Start Blender Bridge:**
   - Open `Sasaki_Visualizer.blend`.
   - Go to Scripture Editor -> Open `blender_receiver.py`.
   - Click "Run Script". The console should say "Listening on :5000".
2. **Start the Brain:**

   ```
   python run_sasaki_live.py --mode production
   ```

3. **Play Music:**
   - Play any audio on the system (Spotify/YouTube).
   - The terminal will show the **KQI Dashboard** (Depth, Quality, State).

10.4 Troubleshooting

10.4.1 Error: "Input Overflow" (PyAudio)

Symptom: The terminal spouts "Input Overflow" and the animation stutters. **Cause:** The `ListenerThread` is processing data slower than the sampling rate (48kHz). This usually happens if `print()` statements are left in the audio callback. **Fix:** Remove all I/O from the callback. Ensure `AudioQ` has a maxsize (e.g., 1024) and drop packets if full.

10.4.2 Error: "Bone Length Explosion"

Symptom: The character's arm stretches to infinity. **Cause:** The GAN generated a NaN (Not a Number) or Infinity value, usually due to a division by zero in the Normalization layer. **Fix:** We implemented `torch.clamp(val, -10, 10)` in `HPI_GCN` output. If this persists, restart the script; the internal LSTM hidden state might be corrupted.

10.4.3 Feature: "The Zombie Drift"

Symptom: The character slowly rotates off-center over 10 minutes. **Cause:** The root velocity accumulation ($P_t = P_{t-1} + V_t$) accumulates floating point error. **Fix:** The system automatically "Center Resets" if audio energy < 0.01 (Silence) for more than 5 seconds. You can force a reset by pressing `R` in the OpenCV window.

Chapter 11: Full API Reference

11.1 gbmmc_engine.py

Class `KQIEngine`

Methods:

- `__init__(self)`: Initializes epsilon.
- `process(r, omega, sigma, t, rho)`: The main calculation loop.
  - **Returns:** Dictionary `{'G', 'Q', 'M', 'C', 'D'}`.
  - **Parameters:**
    - `r` (float): Audio RMS Energy (0.0 - 1.0).
    - `omega` (float): Spectral Flux Variance.
    - `sigma` (float): Signal-to-Noise Ratio inverse.

11.2 sasaki_brain.py

Class `SasakiLiveBrain`

Attributes:

- `self.kqi`: Instance of KQIEngine.
- `self.memory_buffer`: A deque of size 64 storing past latent vectors.
- `self.current_state`: Enum (Flow/Burst/Still).

Methods:

- `decide_style(audio_vector)`:
  - Logic:

1. Check Audio Energy.
2. If Silent -> Apply Heartbeat (Sinewave).
3. Else -> Run KQI.
4. If Depth > 2.0 -> Burst.
5. Else -> Flow.

- **Returns:** `(state, drive, latent_will)`

11.3 realtime_audio_engine.py

Class `RealTimeAudioEngine`

Attributes:

- `self.stream`: PyAudio InputStream.
- `self.buffer`: Rate-limited Queue.

Methods:

- `callback(in_data, frame_count, time_info, status)`:
  - Non-blocking callback. Captures raw bytes, converts to Int16, normalizes to Float32 (-1.0 to 1.0).
- `get_features()`:
  - Runs Librosa MFCC extraction on the last 2048 samples.
  - Apply Z-Score Normalization using a running mean/std buffer.

11.4 blender_receiver.py (Python-in-Blender)

Operator `ModalTimerOperator`

This class hooks into Blender's internal event loop.

- `modal(self, context, event)`: Called every frame.
  1. Polls UDP socket (Non-blocking).
  2. If data: Unpacks 75 floats.
  3. Updates `bpy.data.objects['Bone_Target'].location`.

4. Calls `bpy.context.view_layer.update()` to refresh the viewport.

Chapter 12: The Bug Chronicles (Development Log)

This chapter details the specific, critical failures encountered during the development of the Sasaki-GAN, serving as a technical reference for common pitfalls in Neuro-Symbolic Dance Generation.

12.1 The "Frozen Epoch" (Epoch 0-10)

Date: November 4, 2025 **Symptom:** The Loss curve for the Generator remained exactly at `0.693` (ln 2) while the Discriminator dropped to `0.0`. **Error Log:**

```
Epoch 10/50
Settings: L_adv=1.0, L_phys=0.0
Generator Loss: 0.6931 (Static)
Discriminator Loss: 0.0000 (Perfect)
WARNING: Gradient Norm for Generator is 0.00000e+00
```

Diagnosis: Vanishing Gradients. The Discriminator was perfect too early. **Fix:** implemented `Spectral Normalization` in the Discriminator and added `Gaussian Noise` ($\sigma = 0.1$) to the real inputs.

12.2 The "Sliding Foot" (Epoch 25)

Date: December 12, 2025 **Symptom:** The character performed a Toprock, but the global root position drifted 2 meters to the left. **Code Trace:** In `sasaki_brain.py`:

```
# Old Logic
root_pos += predicted_velocity * delta_t
```

The `predicted_velocity` had a constant bias of `0.001` due to ReLU activation causing a positive mean shift. **Fix:** Switched to LeakyReLU (negative slope 0.2) to center the distribution at zero.

12.3 The "Memory Leak" (Live Run)

Date: January 5, 2026 **Symptom:** After 4 minutes of playback, the FPS dropped from 30 to 5. **Error Log:**

```
RuntimeError: CUDA out of memory. Tried to allocate 20.00 MiB (GPU 0; 8.00 GiB
total capacity; 7.20 GiB already allocated)
```

Diagnosis: The `memory_buffer` list in Python was appending tensors that were still attached to the Computation Graph (`requires_grad=True`). **Fix:** Added `.detach().cpu().numpy()` before taking snapshots for the history buffer.

12.4 The "UDP Packet Loss" (Blender)

Date: January 15, 2026 **Symptom:** The Blender character would "teleport" 10 frames into the future. **Diagnosis:** We were sending packets faster than Blender's Modal Operator could tick. The OS network buffer was filling up, and Blender was reading the *oldest* packet first (FIFO). **Fix:** Changed behavior to LIFO (Last In First Out). The Blender script now drains the entire socket buffer and only processes the *last* received packet.

12.5 The "Silence Panic"

Date: January 20, 2026 **Symptom:** When the music stopped, the GAN outputted random high-frequency noise. **Reason:** Z-Score Normalization divided by zero (std_dev = 0). **Fix:** Added `epsilon=1e-6` to the denominator and a "Silence Gate" that forces the output vector to `zeros` if RMS < 0.001.

Appendix B: Supplementary Code Repository

B.1 Data Augmentation (augment_data.py)

This script applies random rotation (Y-axis) and mirroring to the dataset to effectively quadruple the training data size.

```
def augment_data(data):
    # 1. Mirroring
    mirrored = data.copy()
    mirrored[:, :, 0] *= -1 # Flip X
    # Swap Left/Right Joints (Indices based on NTU)
    mirrored = swap_joints(mirrored, left=[5,6,7], right=[9,10,11])

    # 2. Rotation
    rotated = []
    for angle in [90, 180, 270]:
        r_data = rotate_y(data, angle)
        rotated.append(r_data)

    return np.concatenate([data, mirrored] + rotated)
```

B.2 COCO-17 Graph Definition (coco_17.py)

The minimal graph structure used for the Zero-Padding bridge.

```
# Edge List for COCO-17
edges = [
    (0, 1), (0, 2), # Nose to Eyes
    (1, 3), (2, 4), # Eyes to Ears
    (5, 7), (7, 9), # Left Arm
    (6, 8), (8, 10), # Right Arm
    (11, 13), (13, 15), # Left Leg
    (12, 14), (14, 16), # Right Leg
    (5, 6), (11, 12), # Shoulder/Hip Bridges
    (5, 11), (6, 12)  # Torso
]
```

B.3 3D Visualization Tool (`visualize_3d.py`)

Used for offline verification using Matplotlib.

```
def plot_3d(pose, ax):
    ax.cla()
    for edge in edges:
        p1 = pose[:, edge[0]]
        p2 = pose[:, edge[1]]
        ax.plot([p1[0], p2[0]], [p1[1], p2[1]], [p1[2], p2[2]], 'b-')
    ax.set_xlim(-1, 1)
    ax.set_ylim(-1, 1)
    ax.set_zlim(-1, 1)
```

Chapter 13: Complete Source Code Repository

This chapter contains the complete source code for the critical components of the system.

File: gbmc_engine.py

Location: `c:\Users\kos04\OneDrive\Desktop\vidz\GCN\HPI-GCN\gbmc_engine.py`

```
import math
import random
import time
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional, Tuple


# ============================================================
# 0) 前提: Minimal Buffer Implementation (仮置き)
# ============================================================
@dataclass
class Item:
    payload: Any
    tags: List[str]
    arousal: float
    novelty: float
    confidence: float
    timestamp: float = field(default_factory=time.time)

    def as_dict(self):
        return {
            "payload": self.payload,
            "tags": self.tags,
            "metrics": {"arousal": self.arousal, "novelty": self.novelty, "conf":
self.confidence},
            "ts": self.timestamp
        }

class Buffer:
    def __init__(self, cap: int = 64):
        self.cap = cap
        self.data: List[Item] = []

    def push(self, payload, tags=None, arousal=0.0, novelty=0.0, confidence=0.5):
        it = Item(payload, tags or [], arousal, novelty, confidence)
```

```
        self.data.append(it)
        if len(self.data) > self.cap:
            self.data.pop(0)
        return it

    def view(self, n=10):
        return self.data[-n:]

# ============================================================
# 1) KQI Engine (The Logic Core)
# ============================================================
class KQIEngine:
    """GSMC理論に基づく計算エンジン"""
    def __init__(self):
        self.EPSILON = 1e-9

    def process(self, r: float, omega: float, sigma: float, t: int, rho: float,
theta: float = 1.0, k_mode='inf'):
        # 1. Genesys
        G = r * omega #omega = minesweeper's landmine

        # 2. Base (Sigma check)
        safe_sigma = max(sigma, 0.1) # 0.1を下限として安全マージン確保

        # 3. Quality
        Q = (G ** 2) / safe_sigma

        # 4. Motion (Gate)
        M = 1.0 if Q >= theta else 0.0 #theta = minesweeper's threshold

        # 5. Coherence
        tau = t * rho
        if k_mode == 'inf':
            # tauが大きすぎるとexpが爆発するのでクリップするか、対数スケールで調整
            # ここでは味付けとして tau * 0.1 を入力とする
            C = math.exp(min(tau * 0.1, 10.0))
        elif k_mode == 1:
            C = 1.0 + tau
        else:
            C = 1.0

        # 6. Depth
        D = Q * C * M

        return {"G": G, "Q": Q, "M": M, "C": C, "D": D}

# ============================================================
# 2) MineLife (The Body / Eco-system)
#    (User's provided code, slightly compacted)
# ============================================================
@dataclass
class MineLifeConfig:
    w: int = 16
    h: int = 16
    mine_ratio: float = 0.12
    birth: Tuple[int, ...] = (3,)
    survive: Tuple[int, ...] = (2, 3)
```

```
    reveal_per_tick: int = 2
    danger_spread: float = 0.35
    seed: Optional[int] = None

class MineLife: #if NTU use, embeddied here.
    def __init__(self, cfg: MineLifeConfig):
        self.cfg = cfg
        if cfg.seed is not None: random.seed(cfg.seed)
        self.t = 0
        self.alive = [[1 if random.random() < 0.25 else 0 for _ in range(cfg.w)]
for _ in range(cfg.h)]
        self.mine = [[1 if random.random() < cfg.mine_ratio else 0 for _ in
range(cfg.w)] for _ in range(cfg.h)]
        self.revealed = [[0 for _ in range(cfg.w)] for _ in range(cfg.h)]
        self.risk = [[0.0 for _ in range(cfg.w)] for _ in range(cfg.h)]
        self._update_risk()

    def _n8(self, y, x):
        out = []
        for dy in (-1,0,1):
            for dx in (-1,0,1):
                if dy==0 and dx==0: continue
                ny, nx = y+dy, x+dx
                if 0<=ny<self.cfg.h and 0<=nx<self.cfg.w: out.append((ny, nx))
        return out

    def _count(self, grid, y, x):
        return sum(grid[ny][nx] for (ny, nx) in self._n8(y, x))

    def _update_risk(self):
        base = [[min(1.0, self._count(self.mine, y, x)/8.0) for x in
range(self.cfg.w)] for y in range(self.cfg.h)]
        new_r = [[0.0]*self.cfg.w for _ in range(self.cfg.h)]
        for y in range(self.cfg.h):
            for x in range(self.cfg.w):
                neigh = self._n8(y, x) #if GCN use, embeddied here.
                avg = sum(base[ny][nx] for ny, nx in neigh)/max(1, len(neigh))
                new_r[y][x] = base[y][x]*(1.0-self.cfg.danger_spread) +
avg*self.cfg.danger_spread
        self.risk = new_r

    def _life_step(self):
        nxt = [[0]*self.cfg.w for _ in range(self.cfg.h)]
        for y in range(self.cfg.h):
            for x in range(self.cfg.w):
                n = self._count(self.alive, y, x)
                state = self.alive[y][x]
                nxt[y][x] = 1 if (state==1 and n in self.cfg.survive) or (state==0
and n in self.cfg.birth) else 0
        self.alive = nxt

    def _auto_reveal(self):
        cand = [(y, x) for y in range(self.cfg.h) for x in range(self.cfg.w) if
self.revealed[y][x] == 0]
        if not cand: return []
        cand.sort(key=lambda p: self.risk[p[0]][p[1]]) # Sort by risk (safer first)
        picks = []
```

```
        for _ in range(min(self.cfg.reveal_per_tick, len(cand))):
            # 30% chance to pick from high risk (tail), 70% from low risk (head)
            p = cand.pop(-1) if random.random() > 0.7 else cand.pop(0)
            self.revealed[p[0]][p[1]] = 1
            picks.append((p[0], p[1], self.mine[p[0]][p[1]], self._count(self.mine,
p[0], p[1])))
        return picks

    def tick(self) -> Dict[str, Any]:
        self.t += 1
        self._life_step()
        self._update_risk()
        reveals = self._auto_reveal()

        alive_ratio = sum(sum(r) for r in self.alive) / (self.cfg.w * self.cfg.h)
        revealed_ratio = sum(sum(r) for r in self.revealed) / (self.cfg.w *
self.cfg.h)
        risk_mean = sum(sum(r) for r in self.risk) / (self.cfg.w * self.cfg.h)
        mine_hits = sum(1 for (_, _, hit, _) in reveals if hit == 1)

        # MineLife Internal Metrics
        tension = min(1.0, 0.15 + 0.6 * risk_mean + 0.25 * mine_hits)
        novelty = max(0.0, 1.0 - revealed_ratio) * 0.7 + abs(0.5 - alive_ratio) *
0.3

        tags = ["MineLife"]
        if tension > 0.7: tags.append("high_tension")
        if alive_ratio > 0.55: tags.append("dense_motion")

        return {
            "t": self.t,
            "alive_ratio": alive_ratio,
            "revealed_ratio": revealed_ratio,
            "risk_mean": risk_mean,
            "tension": tension,
            "novelty": novelty,
            "reveals": reveals,
            "tags": tags,
            "mine_hits": mine_hits
        }

# ============================================================
# 3) The Integration: KQI-Powered Brain Buffer
# ============================================================
class BrainBufferOneFile:
    """
    MineLife (無意識/身体) -> KQI Engine (意識/論理) -> Buffer (記憶)
    """
    def __init__(self, buffer_cap: int = 64, mine_cfg: Optional[MineLifeConfig] =
None):
        self.buf = Buffer(cap=buffer_cap)

        self.mine = MineLife(mine_cfg or MineLifeConfig())

        self.kqi = KQIEngine() # ここにKQIエンジンを搭載

        # KQIパラメータ設定
```

```
        self.kqi_theta = 0.5  # Motion閾値
        self.kqi_mode = 'inf' # 共鳴モード

    def tick(self, n: int = 1) -> List[Dict[str, Any]]:
        outs = []
        for _ in range(max(0, n)):
            # 1. MineLifeの状態更新 (Body Update)
            s = self.mine.tick()

            # 2. 変数マッピング (Sensory Input -> KQI Variables)
            # r (Range/Power): 生命密度が高いほどパワーがある (0.0 - 10.0 scale)
            r_val = s["alive_ratio"] * 10.0

            # omega (Velocity): 新規性が高い（変化が激しい）ほど回転が速い (0.0 - 5.0
scale)
            omega_val = s["novelty"] * 5.0

            # sigma (Noise): リスクが高い、または地雷を踏んだ瞬間に急増する
            sigma_val = s["risk_mean"] * 5.0 + (s["mine_hits"] * 10.0)

            # t, rho: 時間と密度
            t_val = s["t"]
            rho_val = s["alive_ratio"]

            # 3. KQI Engine Process (Cognitive Processing)
            kqi_res = self.kqi.process(
                r=r_val,
                omega=omega_val,
                sigma=sigma_val,
                t=t_val,
                rho=rho_val,
                theta=self.kqi_theta,
                k_mode=self.kqi_mode
            )

            # 4. 統合ペイロードの作成
            # MineLifeの生データ + KQIの解釈データ
            full_payload = {
                "source": "MineLife",
                "raw_stats": {k: s[k] for k in ["t", "alive_ratio", "risk_mean",
"tension"]},
                "kqi_analysis": kqi_res, # ここに D, Q, M が入る
                "events": s["reveals"]
            }

            # タグ付けの強化 (KQIの結果に基づいてタグを追加)
            final_tags = s["tags"] + ["KQI_Processed"]
            if kqi_res["M"] > 0:
                final_tags.append("MOTION_ACTIVE")
            if kqi_res["D"] > 100.0:
                final_tags.append("DEEP_RESONANCE")

            # 5. BufferへPush (Memory Storage)
            # KQIの算出値(D)を arousal (覚醒度) として扱うのが自然
            self.buf.push(
                payload=full_payload,
                tags=final_tags,
```

```
                arousal=min(1.0, kqi_res["D"] / 50.0), # Dを正規化して覚醒度へ
                novelty=s["novelty"],
                confidence=kqi_res["M"] # Motionゲートが開いている＝確信がある
            )

            outs.append({"tick": s["t"], "kqi": kqi_res, "tags": final_tags})

        return outs

    def view(self, n: int = 5):
        return [it.as_dict() for it in self.buf.view(n)]

# ============================================================
# テスト実行
# ============================================================
if __name__ == "__main__":
    brain = BrainBufferOneFile(mine_cfg=MineLifeConfig(w=10, h=10,
mine_ratio=0.15))

    print("--- 脳内シミュレーション開始 (KQI Integrated) ---")

    # 10ターン回してみる
    results = brain.tick(n=10)

    for res in results:
        kqi = res["kqi"]
        print(f"Tick {res['tick']:02d} | "
              f"G:{kqi['G']:.2f} σ:{kqi['Q']:.2f} " # Qは便宜上sigmaの逆数的な位置づけだがここではQを表示
              f"-> Motion:{kqi['M']} "
              f"-> Depth(D):{kqi['D']:.4f} "
              f"| Tags: {res['tags']}")
```

File: sasaki_brain.py

Location: c:\Users\kos04\OneDrive\Desktop\vidz\GCN\HPI-GCN\sasaki_brain.py

```
import torch
import numpy as np
import torch.nn.functional as F
import math
import random
from dataclasses import dataclass, field

# --- Partner's Core Theory Implementation ---
class KQIEngine:
    def __init__(self, theta=0.1): # Lowered threshold
        self.theta = theta

    def process(self, r, omega, sigma, t, rho):
        # SENSITIVITY CALIBRATION
        # Add a baseline 0.1 to G so the gate is never permanently locked
        G = (r + 0.1) * (omega + 0.1) * 2.0
        safe_sigma = max(sigma, 0.1)
        Q = (G ** 2) / safe_sigma

        M = 1.0 if Q >= self.theta else 0.2 # Baseline 20% activity
        tau = t * rho
        C = math.exp(min(tau * 0.1, 2.0))

        # --- THE FIX: BASELINE DEPTH ---
        D = (Q * C * M) + 0.1 # Minimum depth 0.1 to keep dancer alive

        return {"G": G, "Q": Q, "M": M, "C": C, "D": D}

class MineLife:
    def __init__(self, w=10, h=10):
        self.w, self.h = w, h
        self.alive = [[1 if random.random() < 0.2 else 0 for _ in range(w)] for _
in range(h)]
        self.t = 0

    def tick(self):
        self.t += 1
        # Simplified cellular automata to simulate "Neural Firing"
        alive_ratio = sum(sum(r) for r in self.alive) / (self.w * self.h)
        novelty = abs(math.sin(self.t * 0.05)) # Slower curiosity drift
        return {"alive_ratio": alive_ratio, "novelty": novelty, "t": self.t}

class SasakiLiveBrain:
    def __init__(self):
        self.current_state = 0 # 0:TR, 1:FW, 2:PW, 3:IDLE
        self.kqi = KQIEngine(theta=0.1) # High sensitivity
```

```
        self.latent_will = np.random.randn(128)
        self.t = 0

    def decide_style(self, audio_vector):
        self.t += 1
        # 1. PERCEPTUAL ENERGY GATE
        energy = audio_vector[-1] # Assuming Onset is last

        # --- THE HEARTBEAT FIX ---
        # If energy is near 0, provide a fake 'Subconscious' pulse
        # This keeps the brain 'alive' even in silence
        if energy < 0.01:
            # Use sine wave to create breathing rhythm
            energy = 0.02 * (1.0 + math.sin(self.t * 0.1))

        # 2. LOGIC CALCULATION (KQI)
        # Non-linear gain: energy squared makes loud music 'explosive'
        r_val = (energy ** 2) * 50.0 # Increased gain
        omega_val = np.var(audio_vector[:20]) * 20.0

        kqi_res = self.kqi.process(r=r_val, omega=omega_val, sigma=0.1, t=self.t,
rho=0.5)

        # FORCE DEPTH TO BE VISIBLE
        kqi_res["D"] = max(kqi_res["D"], 0.1) # Minimum 0.1

        # 3. STOCHASTIC DECISION
        # Fresh random will for every move to break the 'same posture' lock
        self.latent_will = np.random.randn(128)

        if kqi_res["D"] > 2.0:
            self.current_state = np.random.choice([0, 1, 2], p=[0.2, 0.4, 0.4])
        else:
            self.current_state = 0 # Default to Toprock for mid-energy

        drive = np.clip(kqi_res["D"] / 2.0, 1.0, 5.0)
        return self.current_state, drive, kqi_res, self.latent_will
```

File: realtime_audio_engine.py

Location: `c:\Users\kos04\OneDrive\Desktop\vidz\GCN\HPI-GCN\realtime_audio_engine.py`

```python
import numpy as np
import librosa

class RealTimeAudioEngine:
    def __init__(self, sr=15360):
        self.sr = sr

    def get_precise_features(self, audio_chunk):
        """
        Extracts High-Fidelity Audio DNA for Logic (Vector) and UI (Matrix).
        """
        # 0. Safety Checks
        if len(audio_chunk) < 100: return np.zeros(33), np.zeros((20, 1))

        # 1. HANDLE STEREO & GAIN (15x Boost for Intel Arrays)
        if audio_chunk.ndim > 1:
            y = np.mean(audio_chunk, axis=1).flatten().astype(np.float32)
        else:
            y = audio_chunk.flatten().astype(np.float32)
        y = y * 15.0

        # 2. Check for signal presence
        rms = np.sqrt(np.mean(y**2))
        if rms < 0.00005:
             return np.zeros(33), np.zeros((20, 1))

        try:
            # 3. Spectral Analysis (20 MFCCs)
            mfcc = librosa.feature.mfcc(y=y, sr=self.sr, n_mfcc=20)

            # Precise Normalization: Z-score scaling
            # This makes the texture 'dance' in the UI
            mfcc_norm = (mfcc - np.mean(mfcc)) / (np.std(mfcc) + 1e-6)

            # 4. Temporal Compression for Logic
            mfcc_vector = np.mean(mfcc_norm, axis=1)

            # 5. Multi-modal stack (33-dim)
            chroma = np.mean(librosa.feature.chroma_stft(y=y, sr=self.sr), axis=1)
            onset = np.mean(librosa.onset.onset_strength(y=y, sr=self.sr))

            # Return both the Vector (for Brain) and Matrix (for UI)
            return np.hstack([mfcc_vector, chroma, onset]), mfcc_norm

        except Exception as e:
            return np.zeros(33), np.zeros((20, 1))
```

```
    def get_features(self, audio_chunk):
        # Wrapper for backward compatibility with Brain Thread
        vec, _ = self.get_precise_features(audio_chunk)
        return vec
```

File: blender_receiver.py

Location: `c:\Users\kos04\OneDrive\Desktop\vidz\GCN\HPI-GCN\blender_receiver.py`

```
import bpy
import socket
import struct
import mathutils
import os

# --- SETTINGS ---
UDP_IP = "127.0.0.1"
UDP_PORT = 5000
JOINT_COUNT = 25
SCALE_FACTOR = 1.0

# Path to your refinement GLB (Absolute Path to UltraShape Source)
GLB_PATH = r"C:\Users\kos04\OneDrive\Desktop\vidz\GCN\UltraShape-1.0\outputs\refine_demo\charachter_refined.glb"

class UpdReceiverModal(bpy.types.Operator):
    """Runs a modal timer to listen for UDP packets"""
    bl_idname = "wm.udp_receiver_modal"
    bl_label = "Sasaki UDP Receiver"

    _timer = None
    _sock = None

    def modal(self, context, event):
        if event.type == 'TIMER':
            try:
                data, addr = self._sock.recvfrom(1024)
                if len(data) >= 300:
                    count = len(data) // 4
                    floats = struct.unpack(f'{count}f', data)

                    # Update Blender Debug Spheres
                    for i in range(JOINT_COUNT):
                        if i*3 + 2 < len(floats):
                            # HPI-GCN output is typically (X, Y, Z) normalized [-1, 1]
                            # Blender is Z-Up. We usually map:
                            # Python X -> Blender X
                            # Python Y -> Blender Z (Height)
                            # Python Z -> Blender Y (Depth)
                            x = floats[i*3] * SCALE_FACTOR
                            y = floats[i*3+1] * SCALE_FACTOR # Height
                            z = floats[i*3+2] * SCALE_FACTOR # Depth

                            # Update Empty position
```

```
                            obj_name = f"J{i}"
                            if obj_name in bpy.data.objects:
                                obj = bpy.data.objects[obj_name]
                                # Correct Mapping for Blender Viewport
                                obj.location = (x, z, y)

                except BlockingIOError:
                    pass
                except Exception as e:
                    print(f"UDP Error: {e}")

        elif event.type == 'ESC':
            return self.cancel(context)

        return {'PASS_THROUGH'}

    def execute(self, context):
        # 0. Load the UltraShape Character if not present
        if not any("charachter" in o.name for o in bpy.data.objects):
            print(f"📂 Loading UltraShape: {GLB_PATH}")
            if os.path.exists(GLB_PATH):
                try:
                    bpy.ops.import_scene.gltf(filepath=GLB_PATH)
                    print("✅ UltraShape Loaded.")
                except RuntimeError as e:
                    print(f"⚠️ GLB Import Failed (Likely Corrupted): {e}")
                    print("➡️ Proceeding to start UDP Receiver anyway (Debug
Spheres only).")
            else:
                print(f"⚠️ GLB Not Found: {GLB_PATH}")

        # 1. Setup Socket
        self._sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self._sock.bind((UDP_IP, UDP_PORT))
        self._sock.setblocking(False)

        # 2. Setup Debug Skeleton Handles
        collection = bpy.data.collections.get("Sasaki_Rig")
        if not collection:
            collection = bpy.data.collections.new("Sasaki_Rig")
            bpy.context.scene.collection.children.link(collection)

        for i in range(JOINT_COUNT):
            name = f"J{i}"
            if name not in bpy.data.objects:
                bpy.ops.object.empty_add(type='SPHERE', radius=0.03)
                obj = bpy.context.active_object
                obj.name = name
                for coll in obj.users_collection:
                    coll.objects.unlink(obj)
                collection.objects.link(obj)

        # 3. Start Timer
        wm = context.window_manager
        self._timer = wm.event_timer_add(0.016, window=context.window)
        wm.modal_handler_add(self)
        print(f"✅ Listening on {UDP_IP}:{UDP_PORT}")
```

```
        return {'RUNNING_MODAL'}

    def cancel(self, context):
        wm = context.window_manager
        wm.event_timer_remove(self._timer)
        self._sock.close()
        print("🛑 UDP Receiver Stopped")
        return {'CANCELLED'}

def register():
    bpy.utils.register_class(UpdReceiverModal)

def unregister():
    bpy.utils.unregister_class(UpdReceiverModal)

if __name__ == "__main__":
    register()
    bpy.ops.wm.udp_receiver_modal()
```

File: RUN_SASAKI_LIVE.py

Location: `c:\Users\kos04\OneDrive\Desktop\vidz\GCN\HPI-GCN\RUN_SASAKI_LIVE.py`

```
import torch

import numpy as np

import cv2

import sounddevice as sd

import queue

import threading

import json

import time
import socket
import struct

from collections import deque

from scipy.spatial.transform import Slerp, Rotation as R

import scipy.interpolate as interpolate

import warnings

warnings.simplefilter("ignore")



from model.generator import AudioAwareGenerator

from realtime_audio_engine import RealTimeAudioEngine

from sasaki_brain import SasakiLiveBrain



# --- SETTINGS ---

DEVICE = torch.device("cuda")

INPUT_DEVICE = 14

SAMPLING_RATE = 48000
```

```
GOLDEN_LIMIT = 150

BONES = [(0,1), (1,20), (20,2), (2,3), (20,4), (4,5), (5,6), (20,8), (8,9), (9,10),
(0,12), (12,13), (13,14), (0,16), (16,17), (17,18)]



class KineticPostProcessor:

    """Solves Size Consistency and Seamless Stitching."""

    def __init__(self, db_poses):

        self.db_poses = db_poses

        spine_lengths = np.linalg.norm(db_poses[:, 20] - db_poses[:, 0], axis=1)

        self.ref_scale = np.mean(spine_lengths)

        print(f"📏 Reference Human Scale established: {self.ref_scale:.4f}")



    def normalize_pose(self, pose):

        """Forces all skeletons to have the same height/size."""

        curr_scale = np.linalg.norm(pose[20] - pose[0])

        if curr_scale < 1e-6: return pose

        return pose * (self.ref_scale / curr_scale)



    def convert_to_global(self, relative_poses):

        """

        CondMDI Requirement: Cumulatively sum displacements to fix the dancer in
world space.

        """

        global_poses = relative_poses.copy()

        # Assume joint 0 is the root

        # We cumulatively sum the X and Z (floor) displacements across the 64
frames

        # Note: Input must be (T, V, C) or similar. Assuming (T, 25, 3)

        if global_poses.ndim == 3: # (T, 25, 3)

            global_poses[:, 0, [0, 2]] = np.cumsum(relative_poses[:, 0, [0, 2]],
axis=0)
```

```
        elif global_poses.ndim == 2: # (25, 3) - Single frame? Cumsum doesn't apply
well.

             pass

        return global_poses



        return pose * (self.ref_scale / curr_scale)


class BracePatternPlanner:

    def __init__(self):

        # Your specific BRACE data counts

        self.patterns = [

            ([0, 2, 1], 172), # TR -> PW -> FW

            ([0, 1], 70),     # TR -> FW

            ([0, 2], 69),     # TR -> PW

            ([0, 1, 2], 60), # TR -> FW -> PW

            ([2], 28),        # PW ONLY

            ([1], 20),        # FW ONLY

        ]

        # Normalize weights

        counts = np.array([p[1] for p in self.patterns])

        self.weights = counts / counts.sum()



    def select_new_story(self):

        # Pick a full sequence 'Story' to follow

        idx = np.random.choice(len(self.patterns), p=self.weights)

        return self.patterns[idx][0]



class UnityStreamer:
    def __init__(self, ip="127.0.0.1", port=5000):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```
        self.addr = (ip, port)

    def send_pose(self, pose_array):
        # pose_array shape: (25, 3)
        # Flatten to 75 floats
        data = pose_array.flatten().tolist()
        # Pack into binary struct (75 floats)
        msg = struct.pack(f'{len(data)}f', *data)
        self.sock.sendto(msg, self.addr)

class SasakiMasterControlCenter:

    def __init__(self):

        print("💉 Building Full Research Suite...")

        self.gen = AudioAwareGenerator(audio_dim=33).to(DEVICE)


self.gen.load_state_dict(torch.load('./pretrained_models/audio_checkpoints/residual
_sasaki_gen_e50.pth'))

        self.gen.eval()



        # Load Dataset

        db = np.load('./data/brace/BRACE_synced_v2.npz')

        self.db_poses = db['x_data'] if 'x_data' in db else db['x_train']

        self.db_poses = self.db_poses.transpose(0, 2, 3, 1, 4).reshape(-1, 75)

        self.db_labels = db['labels'] if 'labels' in db else db['y_data']

        self.db_tensor = torch.from_numpy(self.db_poses).float().to(DEVICE)



        # Calculate Velocity Manifold (Diff between frames)

        db_vel = np.diff(self.db_poses, axis=0, prepend=self.db_poses[0:1])

        self.db_velocity_tensor = torch.from_numpy(db_vel).float().to(DEVICE)



        # Style Buckets

        expanded_labels = np.repeat(self.db_labels, 64)

        self.bucket_indices = {i: np.where(expanded_labels == i)[0] for i in
range(3)}
```

```
        self.mean_pose_raw = np.load('./data/brace/mean_pose.npy')

        self.mean_pose =
torch.from_numpy(self.mean_pose_raw).float().to(DEVICE).view(1, 3, 1, 25, 1)

        self.audio_engine = RealTimeAudioEngine(sr=SAMPLING_RATE)

        self.brain = SasakiLiveBrain()

        self.audio_queue = queue.Queue()



        self.post_processor = KineticPostProcessor(self.db_poses.reshape(-1, 25,
3))



        # --- FLOW BUFFERS ---

        self.move_queue = queue.Queue(maxsize=2)

        self.is_running = True

        self.prev_pose = None

        self.history = deque(maxlen=20)

        self.boredom = 0.0

        self.current_planned_start_idx = 0 # Track for continuity

        self.current_momentum = torch.zeros(1, 75).to(DEVICE)



        # --- PILLAR 2: GLOBAL ROOT VELOCITY ---

        self.world_root_pos = np.array([0.0, 0.0, 0.0])



        # Forensic Buffers

        self.audio_log = deque(maxlen=GOLDEN_LIMIT)

        self.kinetic_log = deque(maxlen=GOLDEN_LIMIT)

        self.kqi_log = deque(maxlen=GOLDEN_LIMIT)

        self.pose_history = deque(maxlen=5)



        self.pattern_planner = BracePatternPlanner()
        self.latest_live_audio = np.zeros(33) # Real-Time Ear Buffer
        self.latest_mfcc_matrix = np.zeros((20, 1)) # Spectral Matrix Buffer
```

```
        self.streamer = UnityStreamer() # UDP Bridge to Blender/Unity

    def get_playback_params(self, audio_vec):
        # energy = onset, complexity = mfcc variance
        energy = audio_vec[-1] # Assuming last dim is energy or similar
        complexity = np.var(audio_vec[:20])

        # 1. STILLNESS (Freeze)
        if energy < 0.01 and complexity < 0.1:
            return 0.0 # Index does not move

        # 2. SPEED (Conductance)
        # High entropy/energy makes moves fast
        speed = np.clip(energy * 2.0 + complexity * 0.5, 0.5, 2.0)
        return speed

    def get_tta_embedding(self, current_frame, total_frames=64, d_model=33):
        """
        Harvey et al. 2020: Time-to-Arrival Embedding.
        """
        tta = total_frames - current_frame
        # Sinusoidal encoding
        indices = torch.arange(0, d_model // 2).float().to(DEVICE)
        div_term = torch.exp(indices * -(np.log(10000.0) / (d_model / 2)))

        z_tta = torch.zeros(1, 1, d_model).to(DEVICE)
        z_tta[0, 0, 0::2] = torch.sin(tta * div_term)
        z_tta[0, 0, 1::2] = torch.cos(tta * div_term)
        return z_tta
```

```
    def capture_snapshot(self):

        filename = f"GOLDEN_5S_{int(time.time())}.json"

        with open(filename, 'w') as f:

            json.dump({"audio_dna": [a.tolist() for a in self.audio_log],
"kinetic_vectors": [k.tolist() for k in self.kinetic_log], "logic_metrics":
list(self.kqi_log)}, f)

        print(f"\n📸 [SNAPSHOT SAVED]: {filename}")



    def draw_precise_mfcc(self, img, mfcc_matrix):
        """
        Draws a professional-grade spectral monitoring zone.
        mfcc_matrix: (20, T) normalized coefficients.
        """
        # Define UI Area: Top Right Sidebar
        start_x, start_y = 820, 50

        # 1. Draw Frequency Bands
        for i in range(20):
            # Calculate intensity for this band
            val = np.mean(mfcc_matrix[i, :]) if mfcc_matrix.shape[1] > 0 else 0
            # Map Z-score (-2 to 2) to brightness (0 to 255)
            intensity = int(np.clip((val + 2) * 64, 0, 255))

            # Color Logic: High frequency (Top) is Cyan, Low (Bottom) is Pink
            color = (255, 255 - intensity, intensity)

            cv2.rectangle(img,
                          (int(start_x + (i * 18)), start_y),
                          (int(start_x + (i * 18) + 15), start_y + 40),
                          color, -1)

        cv2.putText(img, "SPECTRAL PERCEPTION (MFCC 1-20)", (start_x, start_y -
15),
                    1, 0.6, (200, 200, 200), 1)

    def draw_dashboard(self, img, kqi, style_idx, drive, audio_vec):

        cv2.rectangle(img, (800, 0), (1200, 800), (30, 30, 30), -1)

        # 1. SPECTRAL HEATMAP (New Professional Logic)
        self.draw_precise_mfcc(img, self.latest_mfcc_matrix)



        # 2. METRICS

        cv2.putText(img, f"SASAKI DEPTH (D): {kqi.get('D',0):.2f}", (820, 130), 1,
1.2, (0, 255, 255), 2)
```

```
        cv2.putText(img, f"KINETIC DRIVE: {drive:.2f}x", (820, 170), 1, 1.0, (255,
255, 0), 1)

        cv2.putText(img, f"CREATIVE DRIFT: {self.boredom:.2f}", (820, 210), 1, 1.0,
(255, 0, 255), 1)



        # 3. STYLE

        styles = ["TOPROCK", "FOOTWORK", "POWERMOVE", "IDLE"]

        s_idx = style_idx if style_idx < 4 else 3

        for i, name in enumerate(styles):

            color = (0, 255, 0) if i == s_idx else (80, 80, 80)

            indicator = "[X]" if i == s_idx else "[ ]"

            cv2.putText(img, f"{indicator} {name}", (840, 250 + (i*30)), 1, 1,
color, 2)



        return img



    def apply_masked_sum(self, xt, ground_truth_c, mask_m):

        """

        Formula from PDF: xt_tilde = m * c + (1 - m) * xt

        This 'pins' your real breakdancing moves while the AI denoises the
transition.

        """

        # Ensure mask dimensions match

        # mask_m is (T), need (T, 25, 3) or similar broadcasting

        if mask_m.ndim == 1:

            mask_m = mask_m.view(-1, 1, 1).to(DEVICE)

        return (mask_m * ground_truth_c) + ((1 - mask_m) * xt)



    def diffusion_model(self, xt, t, text_p=None):

        """
```

```
        Mockup/Wrapper for the HPI-GCN acting as UNet.
        """
        # Our generator takes (z, audio, label).
        # For this pivotal upgrade, we treat 'xt' as the latent/input.
        # We need to adapt arguments. This is a PLACHOLDER for the full integration.
        # Returning random noise or simplified prediction for now to satisfy the interface.
        return torch.randn_like(xt) * 0.01

    def update_step(self, xt, eps, t):
        """
        Standard DDPM update step (simplified).
        """
        alpha = 0.9 # placeholder
        return (xt - eps) * alpha + torch.randn_like(xt) * 0.001

    def generate_diffusion_bridge(self, start_move, end_move, text_p):
        """
        Algorithm 2: Sampling.
        start_move: 32 frames of Toprock
        end_move: 32 frames of Power
        """
        # 1. Create Canvas (128 frames)
        # [Start Move (32)] + [Gap (64)] + [End Move (32)]
        c = torch.cat([start_move, torch.zeros(64, 25, 3).to(DEVICE), end_move])
        m = torch.cat([torch.ones(32).to(DEVICE), torch.zeros(64).to(DEVICE), torch.ones(32).to(DEVICE)]) # Mask

        # 2. Start with pure noise
```

```
        xt = torch.randn(128, 25, 3).to(DEVICE)

        # 3. Denoising Loop (Simplified for speed)
        # Reducing to 10 steps for realtime plausibility
        for t in reversed(range(10)):
            # Force the AI to see your real moves
            xt_tilde = self.apply_masked_sum(xt, c, m)

            # Predict the 'Clean' motion using text prompt p
            eps = self.diffusion_model(xt_tilde, t, text_p)

            # Update xt-1
            xt = self.update_step(xt, eps, t)

        return xt # The final 128-frame seamless dance

    def bi_objective_rag_search(self, query_intent, anchor_pose, prev_velocity,
style_idx):
        """
        UPGRADED: Sasaki Kinetic Gum Search.
        prev_velocity: (1, 75) tensor representing the 'momentum' of the last move.
        """
        indices = self.bucket_indices[style_idx]
        manifold = self.db_tensor[indices]
        vel_manifold = self.db_velocity_tensor[indices]

        # 1. Position Distance (Shape)
        d_pose = torch.cdist(anchor_pose, manifold)
        # 2. Music Distance (Intent)
```

```
        d_music = torch.cdist(query_intent, manifold)

        # 3. Velocity Distance (Momentum / The 'Gum')

        d_vel = torch.cdist(prev_velocity, vel_manifold)



        # Normalize

        d_pose /= (torch.max(d_pose) + 1e-6)

        d_music /= (torch.max(d_music) + 1e-6)

        d_vel /= (torch.max(d_vel) + 1e-6)



        # --- PHASE 3: CATEGORY TRANSITION PENALTY ---

        velocity_magnitude = torch.norm(prev_velocity)

        if velocity_magnitude > 0.4: # If moving fast

            # High Speed: prioritize the 'swing' (40%) to ensure flow

            total_dist = (0.4 * d_music) + (0.2 * d_pose) + (0.4 * d_vel)

        else:

            # Low Speed: focus on pose shape and music

            total_dist = (0.5 * d_music) + (0.4 * d_pose) + (0.1 * d_vel)



        # 5. STOCHASTIC TOP-K (Prison Break)
        k = 100
        _, top_k_indices = torch.topk(total_dist, k, largest=False)

        # Randomly pick from the top 10% of matches
        random_pick = np.random.randint(0, 10)
        try:
             selection_idx = indices[top_k_indices[0, random_pick].item()]
        except:
             selection_idx = indices[top_k_indices[0, 0].item()]

        # Tabu history...
        self.history.append(selection_idx)
        return selection_idx



    def brain_thread_loop(self):

        print("🧠 Brain Thread: Choreographic Conductor Online.")
```

```
        while self.is_running:
            # 1. Select the 'Story' (Pattern) based on BRACE stats
            current_story = self.pattern_planner.select_new_story()

            for style_idx in current_story:
                # 2. Accumulate audio context specifically for THIS segment
                chunks = []
                while len(chunks) < 4: # Wait for ~0.4s of audio for valid texture
                    try: chunks.append(self.audio_queue.get(timeout=0.5))
                    except queue.Empty: break

                if not chunks: continue
                audio_vec =
self.audio_engine.get_features(np.concatenate(chunks).flatten())

                # 3. Decide Kinetic Drive and Intent
                _, drive, kqi, will = self.brain.decide_style(audio_vec)
                speed = self.get_playback_params(audio_vec)

                # 4. Generate Intent & Bi-Objective Search
                z = torch.from_numpy(will).float().to(DEVICE).unsqueeze(0)
                audio_t =
torch.from_numpy(audio_vec).float().to(DEVICE).view(1,1,33).repeat(1,64,1)
                label_oh = torch.eye(3, device=DEVICE)[style_idx].unsqueeze(0)

                with torch.no_grad():
                    delta = self.gen(z, audio_t, label_oh)
                    push = 15.0 + (drive * 20.0)
                    messy = (self.mean_pose + (delta * push)).squeeze().permute(1,
2, 0).reshape(64, 75)
```

```
                    query_intent = messy[0:1]


                # Momentum-Aware Search
                anchor_pose =
torch.from_numpy(self.prev_pose).float().to(DEVICE).reshape(1, 75) if
self.prev_pose is not None else torch.zeros(1, 75).to(DEVICE)
                global_idx = self.bi_objective_rag_search(query_intent,
anchor_pose, self.current_momentum, style_idx)


                # 5. Push Move to Performance Queue
                self.move_queue.put({
                    "style": style_idx,
                    "global_idx": global_idx,
                    "speed": speed,
                    "kqi": kqi,
                    "audio": audio_vec
                })


    def start(self):
        cv2.namedWindow("SASAKI-GAN CONDUCTOR", cv2.WINDOW_NORMAL)


        # Launch Brain Thread
        threading.Thread(target=self.brain_thread_loop, daemon=True).start()


        # --- FIX: ROBUST DEVICE QUERY ---
        try:
            # Query without kind='input' because Loopback is technically an Output-
Input hybrid
            dev_info = sd.query_devices(INPUT_DEVICE)
            # WASAPI Loopback REQUIRES the hardware's native channel count (usually
2)
            ch = int(dev_info['max_input_channels'])
            if ch == 0: # If it claims 0, it's definitely a loopback of an output
                ch = int(dev_info['max_output_channels'])

            print(f"🎧 Loopback Active: {dev_info['name']}")
```

```
            print(f"📊 Hardware Channels: {ch} | Rate: {SAMPLING_RATE}Hz")
        except Exception as e:
            print(f"⚠️ Device Query Warning: {e}. Defaulting to Stereo.")
            ch = 2

        def audio_callback(indata, frames, time, status):
             self.audio_queue.put(indata.copy())
             # IMMEDIATELY update the live perception for the Conductor
             features, matrix =
self.audio_engine.get_precise_features(indata.flatten())
             self.latest_live_audio = features
             self.latest_mfcc_matrix = matrix

        with sd.InputStream(device=INPUT_DEVICE, samplerate=SAMPLING_RATE,
channels=ch, callback=audio_callback):

            print("🚀 PERFORMANCE ACTIVE. ZERO-LATENCY FLOW.")



            while True:

                # 1. Wait for the Brain to provide a pre-calculated Move

                try:

                    move = self.move_queue.get(timeout=5.0)

                except queue.Empty:

                    print("⚠️ Waiting for Brain Thread...")

                    continue



                global_idx = move["global_idx"]

                speed = move["speed"]

                prev_move_end =
torch.from_numpy(self.prev_pose).float().to(DEVICE).reshape(25, 3) if
self.prev_pose is not None else None



                # 2. Execute the 64-frame sequence with Dynamic Continuity

                f_pointer = 0.0

                while f_pointer < 64:

                    f_idx = int(f_pointer)

                    f_pointer += max(0.2, speed) # Advance pointer (Min 0.2 ensures
move doesn't freeze)
```

```
                    if f_idx >= 64: break

                    # 3. Get Pose & Apply Global Root Velocity (CondMDI Inertial Drift)
                    raw_pose = self.db_poses[(global_idx + f_idx) % len(self.db_poses)].reshape(25, 3)

                    if f_idx == 0 and self.prev_pose is not None:
                        # Align hips to prevent ghost-snapping
                        target_offset = self.prev_pose[0] - raw_pose[0]
                        # EMA Filter for smooth root transition
                        self.root_offset = 0.8 * getattr(self, 'root_offset', target_offset) + 0.2 * target_offset

                    # 4. Kinetic Gum Blending
                    if f_idx < 15 and prev_move_end is not None:
                        m = f_idx / 15.0
                        c_tensor = torch.from_numpy(raw_pose).float().to(DEVICE)
                        render_pose = (m * c_tensor + (1 - m) * prev_move_end).cpu().numpy()
                    else:
                        render_pose = raw_pose

                    # 5. Transform & Velocity Tracking
                    final_output = self.post_processor.normalize_pose(render_pose + getattr(self, 'root_offset', 0))

                    if self.prev_pose is not None:
                        mom = (final_output - self.prev_pose).flatten()
                        self.current_momentum = torch.from_numpy(mom).float().to(DEVICE).unsqueeze(0)
```

```
                    # 6. Render Dashboard & Skeleton (Use LIVE Audio)
                    img = np.zeros((800, 1200, 3), dtype=np.uint8) + 20

                    # Update parameters based on REAL-TIME MIC data
                    live_speed = self.get_playback_params(self.latest_live_audio)
                    # Use live speed for conducting
                    speed = live_speed

                    img = self.draw_dashboard(img, move["kqi"], move["style"],
speed, self.latest_live_audio)

                    # 7. UDP STREAM TO 3D ENGINE
                    self.streamer.send_pose(final_output)

                    coords = (final_output[:, :2] * 450 + 400).astype(int)



                    for u, v in BONES:

                        cv2.line(img, (coords[u,0], coords[u,1]), (coords[v,0],
coords[v,1]), (0, 255, 120), 3, cv2.LINE_AA)



                    cv2.imshow("SASAKI-GAN CONDUCTOR", img)

                    self.prev_pose = final_output.copy()



                    # Manual Kill Switch

                    if cv2.waitKey(30) == ord('q'):

                        self.is_running = False

                        return


if __name__ == "__main__":

    SasakiMasterControlCenter().start()
```

File: audio_brace_loader.py

Location: `c:\Users\kos04\OneDrive\Desktop\vidz\GCN\HPI-GCN\audio_brace_loader.py`

```
import numpy as np
import torch
from torch.utils.data import Dataset
import os
import glob
import pandas as pd

class AudioBraceDataset(Dataset):
    def __init__(self, skeleton_path, audio_dir, fps=30):
        print(f"Loading Synced Skeleton Data from: {skeleton_path}")
        self.skel_data = np.load(skeleton_path)

        self.skeleton_data = self.skel_data['x_data']
        self.video_ids = self.skel_data['video_ids']
        self.seq_indices = self.skel_data['seq_indices']
        self.audio_offsets = self.skel_data['audio_offsets']
        self.labels = self.skel_data['labels']

        # Load FPS Metadata (New)
        if 'fps' in self.skel_data:
            self.fps_data = self.skel_data['fps']
        else:
            print("Warning: FPS metadata not found. Defaulting to 30.")
            self.fps_data = np.full(len(self.video_ids), 30.0)

        self.audio_dir = audio_dir

        print("Indexing segment audio files...")
        self.audio_map = {}
        all_audio_files = glob.glob(os.path.join(audio_dir, "**", "*.npz"),
recursive=True)
        for path in all_audio_files:
            fname = os.path.basename(path)
            parts = fname.replace(".npz", "").split(".")
            if len(parts) >= 2:
                vid_id = parts[0]
                try:
                    seq_idx = int(parts[1])
                    self.audio_map[(vid_id, seq_idx)] = path
                except: continue

        print(f"Found {len(self.audio_map)} mapped audio segments.")

    def __len__(self):
        return len(self.skeleton_data)
```

```
    def __getitem__(self, index):
        skeleton = self.skeleton_data[index]
        label = self.labels[index]

        vid_id = self.video_ids[index]
        seq_idx = self.seq_indices[index]
        audio_offset = self.audio_offsets[index]
        video_fps = self.fps_data[index]

    def __getitem__(self, index):
        skeleton = self.skeleton_data[index]
        label = self.labels[index]

        vid_id = self.video_ids[index]
        seq_idx = self.seq_indices[index]
        audio_offset = self.audio_offsets[index]

        audio_path = self.audio_map.get((vid_id, seq_idx))
        TARGET_FRAMES = 64

        # Audio Sampling Rate Investigation Result:
        # Majority of dataset features are 1:1 mapped to Video Frames.
        # (25fps video -> 25Hz audio, 30fps video -> 30Hz audio).
        # Therefore, direct indexing is the most robust method.
        # Mean Lag was -1.66 frames. Applying +2 correction.

        if audio_path:
            try:
                audio_data = np.load(audio_path)

                mfcc = audio_data['mfcc'].squeeze().T
                chroma = audio_data['chroma_cqt'].squeeze().T
                onset = audio_data['onset_env'].ravel()[:, np.newaxis]
                combined = np.hstack([mfcc, chroma, onset]) # (Total_T, 33)

                # Correction based on Audit v5 (Mean Lag -35)
                # Motion is Early -> We need to shift audio retrieval LATER to
match.
                SYSTEMIC_CORRECTION = 35

                # Raw start index
                raw_start = audio_offset + SYSTEMIC_CORRECTION

                # Safety Clamping (Prevent Crash on Short Audio)
                max_start = len(combined) - TARGET_FRAMES
                start_idx = max(0, min(raw_start, max_start))
                end_idx = start_idx + TARGET_FRAMES

                crop = combined[start_idx : end_idx]

                # Final check for length (padding if absolutely necessary)
                if len(crop) < TARGET_FRAMES:
                    pad_amt = TARGET_FRAMES - len(crop)
                    crop = np.pad(crop, ((0, pad_amt), (0, 0)), mode='constant')

                audio_vector = torch.from_numpy(crop).float()
```

```
            except Exception as e:
                audio_vector = torch.zeros(TARGET_FRAMES, 33)
        else:
            audio_vector = torch.zeros(TARGET_FRAMES, 33)

        skeleton = torch.from_numpy(skeleton).float()
        return skeleton, audio_vector, label



if __name__ == "__main__":
    SKELETON_PATH = './data/brace/BRACE_synced_v2.npz'
    AUDIO_DIR = './brace/audio_features/'

    if os.path.exists(SKELETON_PATH):
        ds = AudioBraceDataset(SKELETON_PATH, AUDIO_DIR)
        print(f"Dataset Length: {len(ds)}")
        s, a, l = ds[0]
        print(f"Sample 0 Shapes - Skeleton: {s.shape}, Audio: {a.shape}")
        print(f"Sample 0 Meta - Vid: {ds.video_ids[0]}, Seq: {ds.seq_indices[0]},
Offset: {ds.audio_offsets[0]}")
    else:
        print(f"Synced dataset not found. Run build_brace_v2.py first.")
```

File: calc_mean_pose.py

Location: `c:\Users\kos04\OneDrive\Desktop\vidz\GCN\HPI-GCN\calc_mean_pose.py`

```python
import numpy as np

DATA_PATH = './data/brace/BRACE_synced_v2.npz'
OUTPUT_PATH = './data/brace/mean_pose.npy'

def calc_mean():
    print(f"Loading {DATA_PATH}...")
    data = np.load(DATA_PATH)
    x_data = data['x_data'] # (N, 3, 64, 25, 1)

    # Calculate Mean across Batch (0) and Time (2)
    # Result: (3, 25, 1) -> (3, 25)
    mean_pose = np.mean(x_data, axis=(0, 2, 4))

    print(f"Mean Pose Shape: {mean_pose.shape}")
    print(f"Sample Joint 0 (Nose): {mean_pose[:, 0]}")

    np.save(OUTPUT_PATH, mean_pose)
    print(f"Saved Mean Pose to {OUTPUT_PATH}")

if __name__ == "__main__":
    calc_mean()
```

File: SASAKI/Improvisation_SLM.py

Location: `c:\Users\kos04\OneDrive\Desktop\vidz\GCN\HPI-GCN\SASAKI/Improvisation_SLM.py`

```python
from dataclasses import dataclass, asdict
from enum import Enum
import math
import random
from typing import List, Dict, Optional


class State(str, Enum):
    F = "F"  # 前進
    B = "B"  # 後退
    S = "S"  # 停滞


@dataclass
class SLMConfig:
    # 自由パラメータ6個（残りは 1 - (p_xy + p_xz)）
    p_ff: float = 0.6
    p_fb: float = 0.2
    p_bf: float = 0.3
    p_bb: float = 0.5
    p_sf: float = 0.4
    p_sb: float = 0.4

    temperature_tau: float = 1.0    # ゆらぎ量
    skill_level: float = 0.5       # 停滞のコントロール力
    learning_rate_eta: float = 0.1 # 学習率

    def transition_matrix(self):
        """
        3x3行列 P[state_from][state_to]
        行の残りは 1 - (p_xy + p_xz) で S行きにする。
        順番は [F, B, S]
        """
        # F行
        p_fs = max(0.0, 1.0 - (self.p_ff + self.p_fb))
        # B行
        p_bs = max(0.0, 1.0 - (self.p_bf + self.p_bb))
        # S行
        p_ss = max(0.0, 1.0 - (self.p_sf + self.p_sb))

        return {
            State.F: {State.F: self.p_ff, State.B: self.p_fb, State.S: p_fs},
            State.B: {State.F: self.p_bf, State.B: self.p_bb, State.S: p_bs},
            State.S: {State.F: self.p_sf, State.B: self.p_sb, State.S: p_ss},
        }
```

```python
@dataclass
class SLMState:
    current_state: State = State.F
    internal_clock: float = 0.0
    last_entropy: float = 0.0
    step_index: int = 0


@dataclass
class LogEntry:
    t: int
    state_from: State
    state_to: State
    entropy: float
    reward: float
    note: str = ""


class ImprovisationSLM:
    """
    3状態マルコフ連鎖 + 温度パラメータ + 熟練度ゲート で構成された
    必要最小限の即興エンジン。
    """

    def __init__(self, config: SLMConfig):
        self.config = config
        self.log: List[LogEntry] = []

    @staticmethod
    def softmax(logits, tau: float):
        if tau <= 0:
            # τ→0 なら argmax に近づける
            max_idx = max(range(len(logits)), key=lambda i: logits[i])
            out = [0.0] * len(logits)
            out[max_idx] = 1.0
            return out

        exps = [math.exp(l / tau) for l in logits]
        s = sum(exps)
        if s == 0:
            return [1.0 / len(logits)] * len(logits)
        return [e / s for e in exps]

    def compute_context_entropy(
        self,
        audio_features,
        pose_features,
    ) -> float:
        """
        シャノンエントロピーの超ざっくり版。
        実際には BRACE の音特徴・ポーズ変化量から確率分布を作って
        H = - Σ p log p を計算するイメージ。
        ここでは placeholder 実装。
        """
        # 例: 正規化エネルギーっぽいものをまとめて 0〜1 に押し込む
        val = float(abs(audio_features)) + float(abs(pose_features))
```

```
        # 適当にスケーリングして 0〜1 くらいに
        return max(0.0, min(1.0, val / 10.0))

    def step(
        self,
        state: SLMState,
        audio_entropy_source,
        pose_entropy_source,
        reward: float = 0.0,
    ) -> SLMState:
        """
        SLM を1ステップ進める。
        - audio_entropy_source / pose_entropy_source は
          BRACEから切ったウィンドウの特徴量から計算した指標を想定。
        """
        cfg = self.config
        P = cfg.transition_matrix()

        # 文脈エントロピー（本当はここでシャノンエントロピーを計算）
        context_entropy = self.compute_context_entropy(
            audio_entropy_source, pose_entropy_source
        )

        # 温度をエントロピーで少し動かす例
        # エントロピーが高いほどバラける
        tau = cfg.temperature_tau * (0.5 + context_entropy)

        # 現状態の遷移確率を logits にして softmax
        row = P[state.current_state]
        states = [State.F, State.B, State.S]
        base_probs = [row[s] for s in states]
        # logを取ることで softmax の入力に
        logits = [math.log(max(p, 1e-8)) for p in base_probs]

        probs = self.softmax(logits, tau)

        # 熟練度で S 行きの確率をゲート
        # skill=0 → Sはほぼ出ない
        # skill=1 → Sの確率そのまま
        idx_S = states.index(State.S)
        probs[idx_S] *= cfg.skill_level

        # 正規化し直す
        s = sum(probs)
        if s > 0:
            probs = [p / s for p in probs]
        else:
            probs = [1.0 / len(probs)] * len(probs)

        # サンプリング
        r = random.random()
        cum = 0.0
        next_state = states[-1]
        for s_state, p in zip(states, probs):
            cum += p
            if r <= cum:
                next_state = s_state
```

```
            break

        # ログ
        self.log.append(
            LogEntry(
                t=state.step_index,
                state_from=state.current_state,
                state_to=next_state,
                entropy=context_entropy,
                reward=reward,
            )
        )

        # 学習（超シンプルな報酬付き微調整；本格的なRLにするならここ拡張）
        if reward != 0.0:
            # 報酬が正 → 選んだ遷移の確率を少し上げる、負 → 下げる
            eta = cfg.learning_rate_eta
            from_row = P[state.current_state]
            old_p = from_row[next_state]
            delta = eta * reward
            new_p = max(0.0, min(1.0, old_p + delta))
            # 簡易的に、選んだ遷移だけ書き換えて normalize は省略
            if state.current_state == State.F and next_state == State.F:
                cfg.p_ff = new_p
            elif state.current_state == State.F and next_state == State.B:
                cfg.p_fb = new_p
            elif state.current_state == State.B and next_state == State.F:
                cfg.p_bf = new_p
            elif state.current_state == State.B and next_state == State.B:
                cfg.p_bb = new_p
            elif state.current_state == State.S and next_state == State.F:
                cfg.p_sf = new_p
            elif state.current_state == State.S and next_state == State.B:
                cfg.p_sb = new_p

        # 状態更新
        return SLMState(
            current_state=next_state,
            internal_clock=(state.internal_clock + 0.25) % 1.0,
            last_entropy=context_entropy,
            step_index=state.step_index + 1,
        )
```